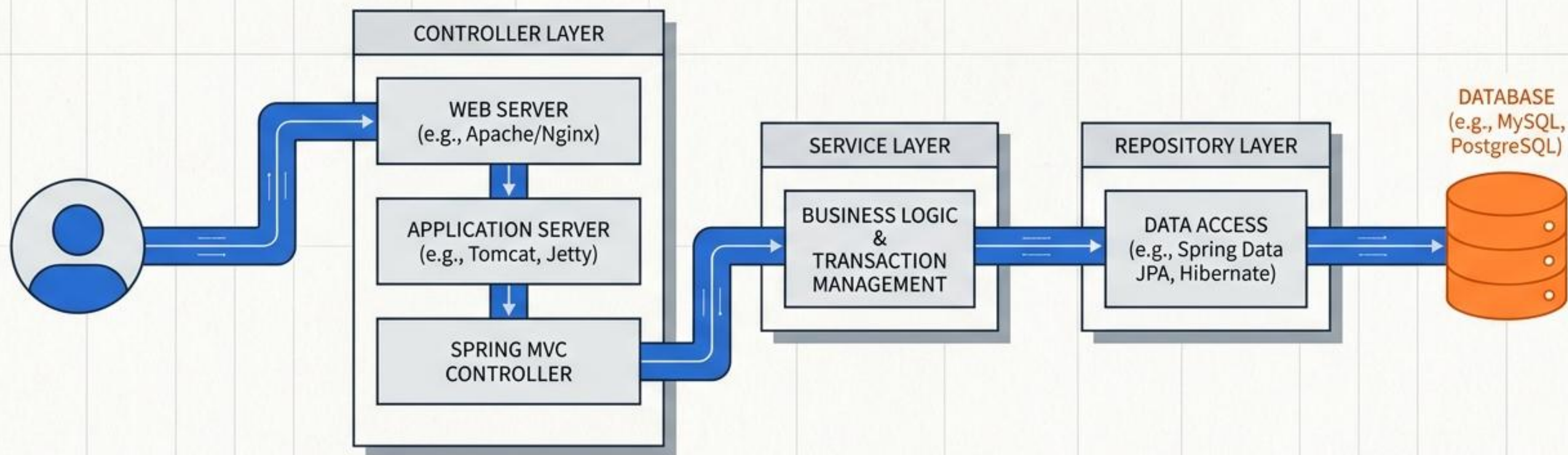


모던 웹 요청의 설계도

Java 웹 백엔드를 구성하는 5가지 핵심 기술과 MVC 아키텍처의 완벽한 이해

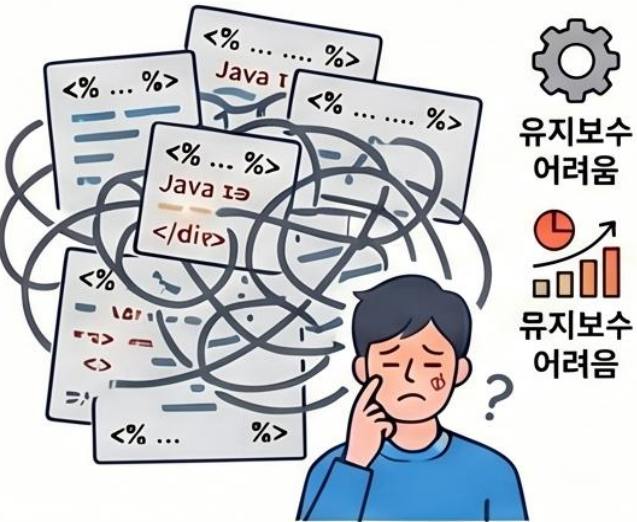


효율적인 웹 개발을 위한 JSP 현대화 & DB 성능 최적화 가이드

JSP의 진화: 스크립트릿에서 EL/JSTL로

스크립트에서, 스크립트릿에서 EL/JSTL, 및 로직 처리화 가입

기존 방식 (구식)



스크립트릿:
복잡한 Java 코드 혼용

현대적 방식 (개선)



EL/JSTL 도입:
Java 코드 없이 데이터 출력 및 로직 처리

코드 직관성 비교

	데이터 출력	현대적 방식
데이터 출력	<code><%= member.getName() %></code>	<code>\${member.name}</code>
반복문 처리	<code><% for(...) { %></code>	<code><c:forEach ...></code>
조건문 처리	<code><% if(age >= 20) { %></code>	<code><c:if test="\${age >= 20}"></code>

View(화면)

Business
Logic

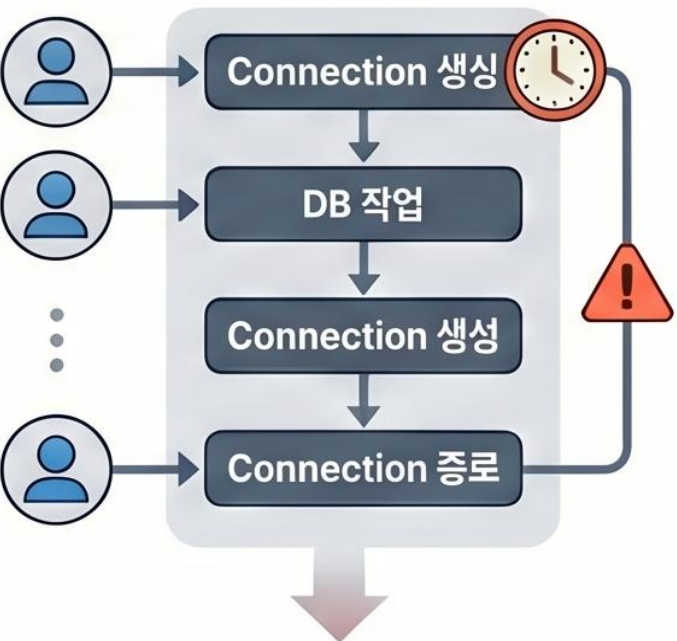
문화, 출력

MVC 역할 혼재

성능 최적화의 핵심: JDBC vs DBCP

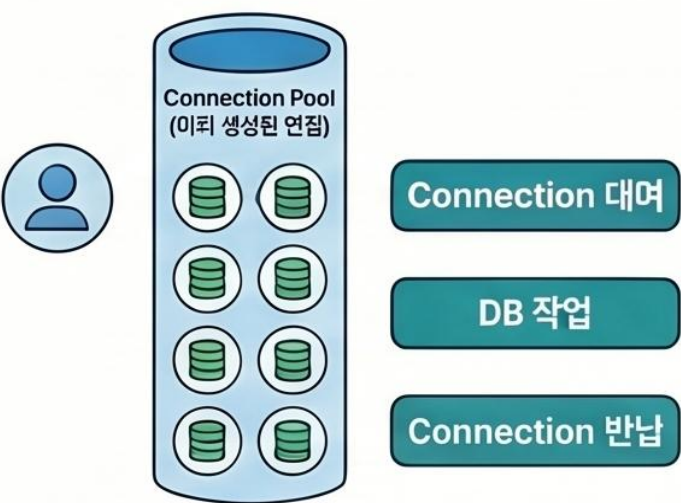
성능 최적화 연결 · JDBC vs DBCP로 위해 연결 가능

JDBC 방식 (비효율)



매 요청마다 생성/종료:
높은 연결 비용, 서버 자원 소모, 속도 저하

DBCP 방식 (효율적)



연결 재사용: 응답 속도 획기적 향상



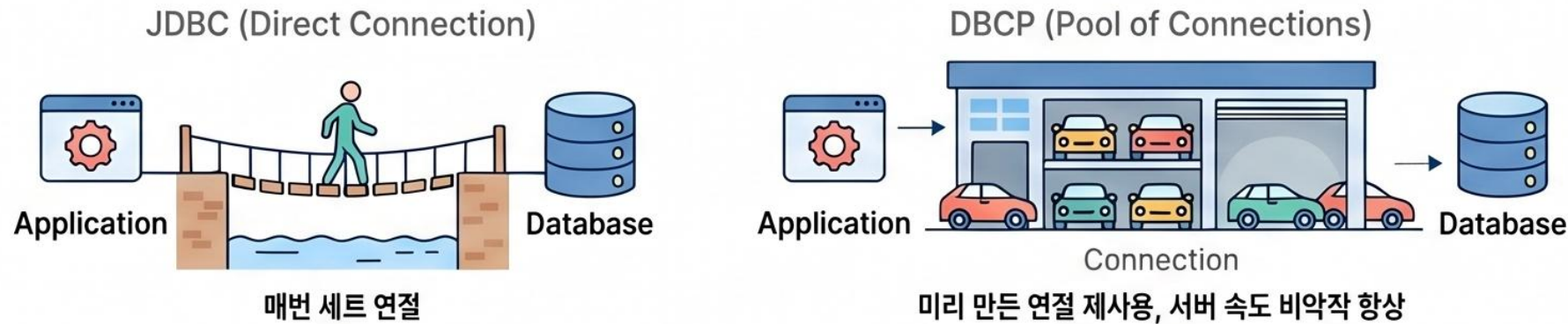
실무 대세: HikariCP
Spring Boot 기본값, 우수한 성능

JSP & 웹 백엔드 핵심 기술 한눈에 보기

JSP 기반 웹 개발의 핵심 기술(JDBC, DBCP, EL/JSTL, MVC)의 개념과 유기적인 동작 흐름

웹 백엔드를 지탱하는 5가지 핵심 기술 🎉

JDBC(다리) vs DBCP(렌터카)



EL & JSTL: JSP에서 Java 코드를 제거하다

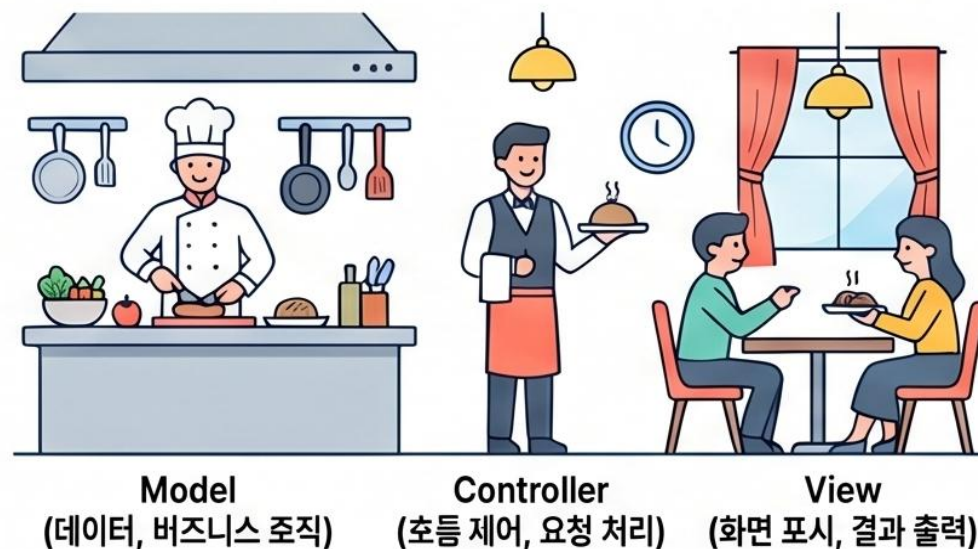


JSP 출력 방식의 보안성과 편의성 차이

방식	특정	보안 (XSS)
<code><%= %></code>	Java 코드 직접 출력 (구식)	❌ 위험
<code>\${ }</code>	깔끔한 값 접근 및 출력	⚠️ 보통
<code><c:out></code>	JSTL 출력 + 특수문자 변환	✅ 안전

MVC 구조: 요리사와 종업원의 분업

HTML 가독성을 높이고 유지보수를 쉽게 만듭니다
역할을 분리하여 개발 효율을 극대화하는 표준 설계 패턴입니다



사용자 요청부터 화면 출력까지의 흐름

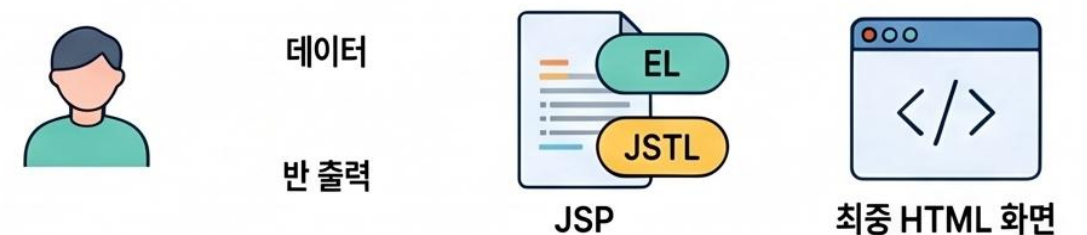
1. 요청 및 제어 (Controller)



2. 데이터 처리 (Model)

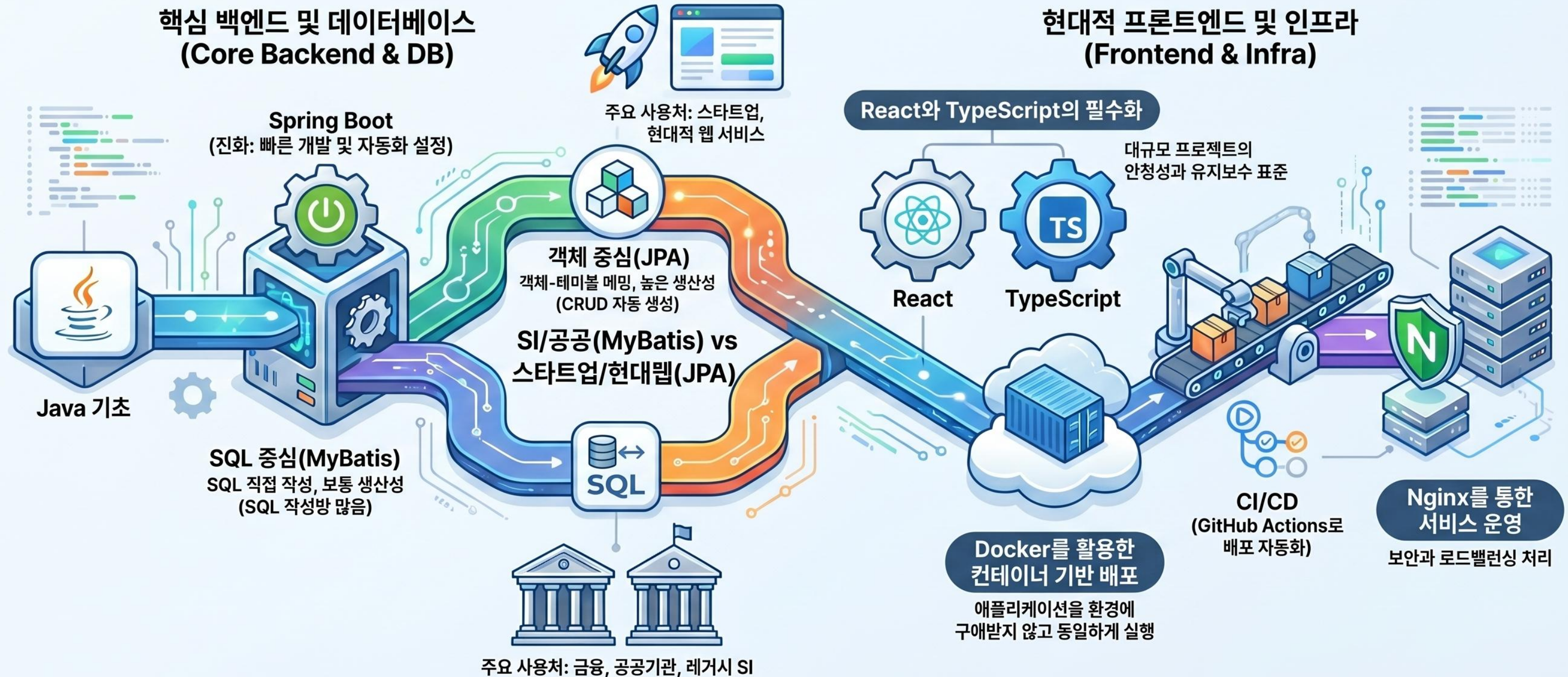


3. 결과 반환 및 출력 (View)



2026 자바 백엔드 개발자 로드맵: 기초부터 실무 트렌드까지

자바 백엔드 개발자가 되기 위해 학습해야 할 기술 스택의 흐름과 실무에서 중요하게 다루는 기술들을 단계별로 안내합니다.



백엔드 물류 센터를 움직이는 5가지 핵심 부품

JDBC



Java가 DB에 접속하는
기본 다리
(데이터 접근)

DBCP



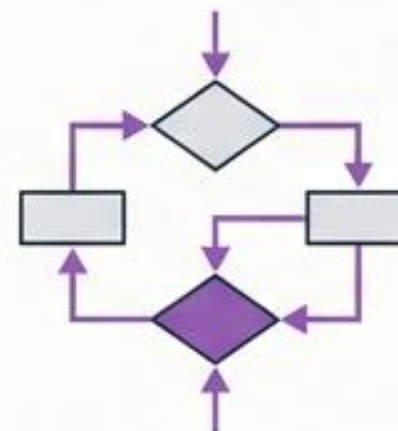
DB 연결을 미리 만들어
재사용하는 풀(Pool)
(속도 향상)

EL



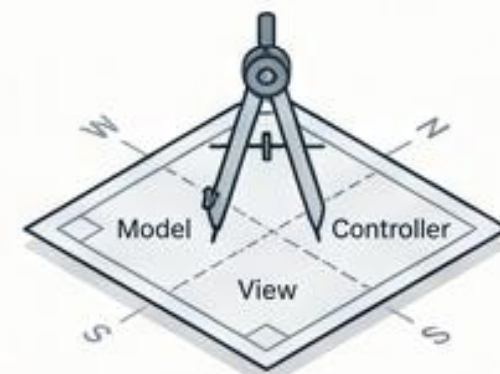
JSP에서 Java 코드 없이
값을 깔끔하게 출력
(데이터 표현)

JSTL



JSP를 HTML처럼,
반복/조건문을 쉽게
(로직 처리)

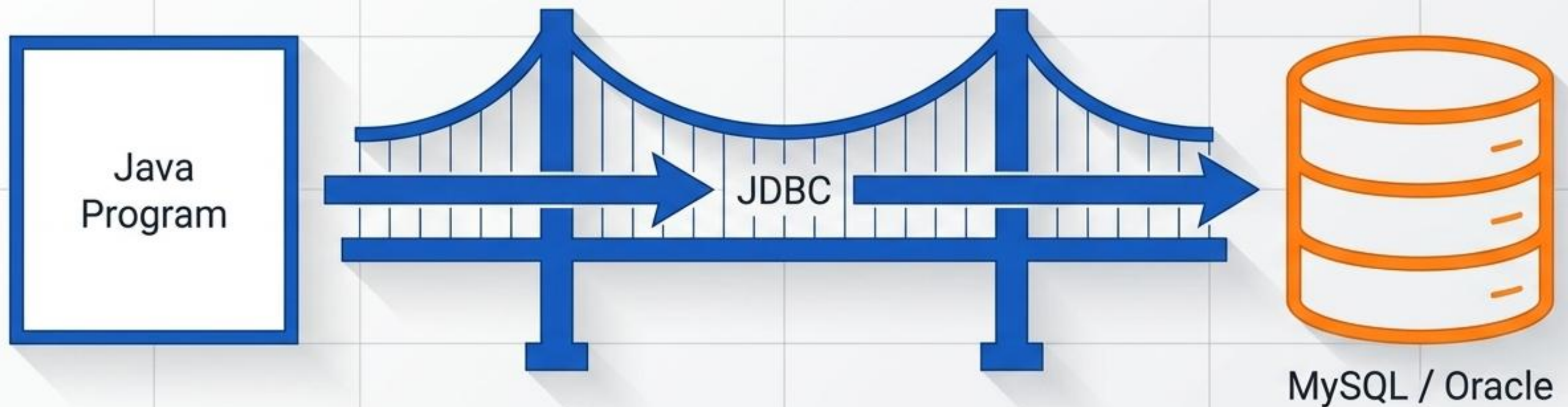
MVC



이 모든 것을 역할별로
나누는 지휘 체계
(설계 패턴)

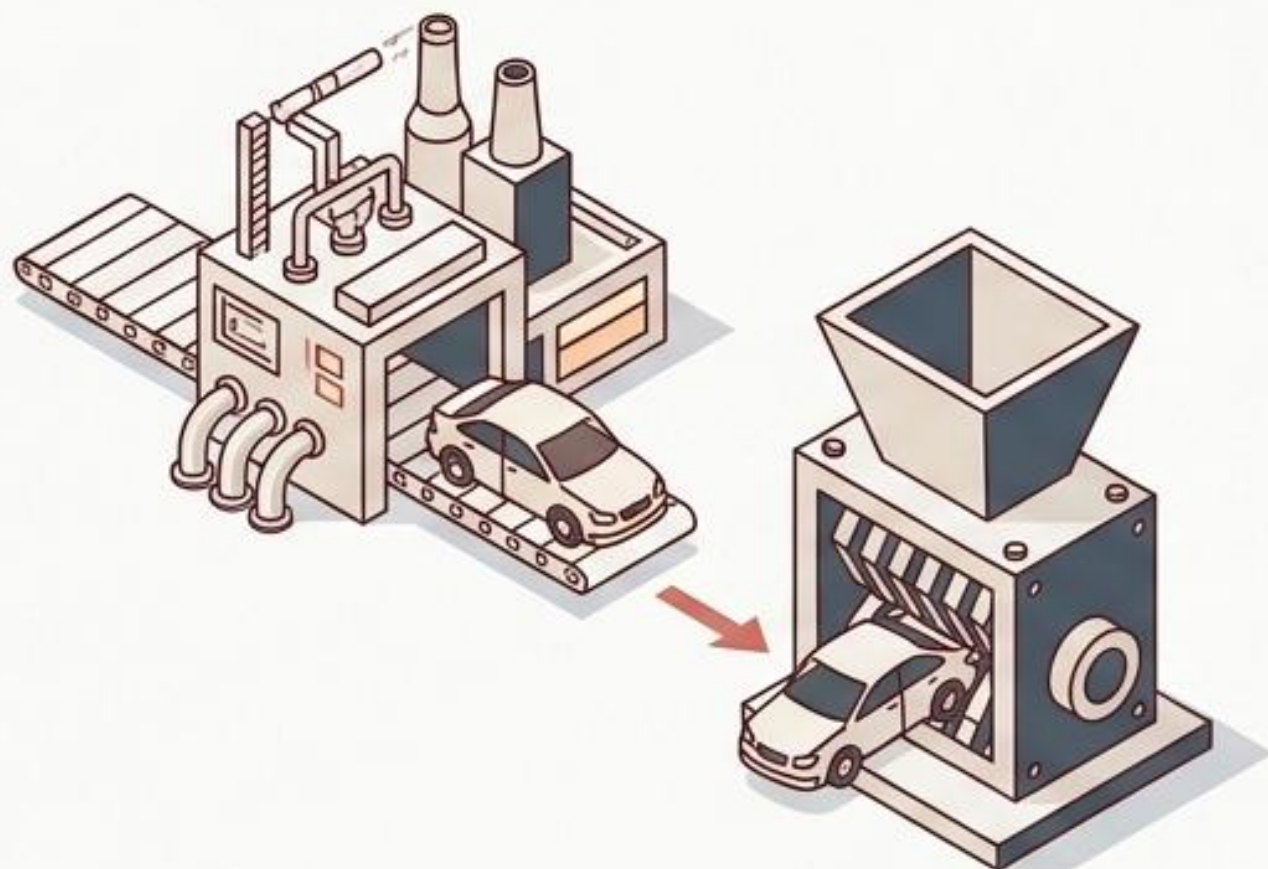
JDBC: 데이터베이스로 향하는 유일한 다리

Java 프로그램이 데이터베이스와 통신하기 위한 필수 기술 (Connection, PreparedStatement, ResultSet)



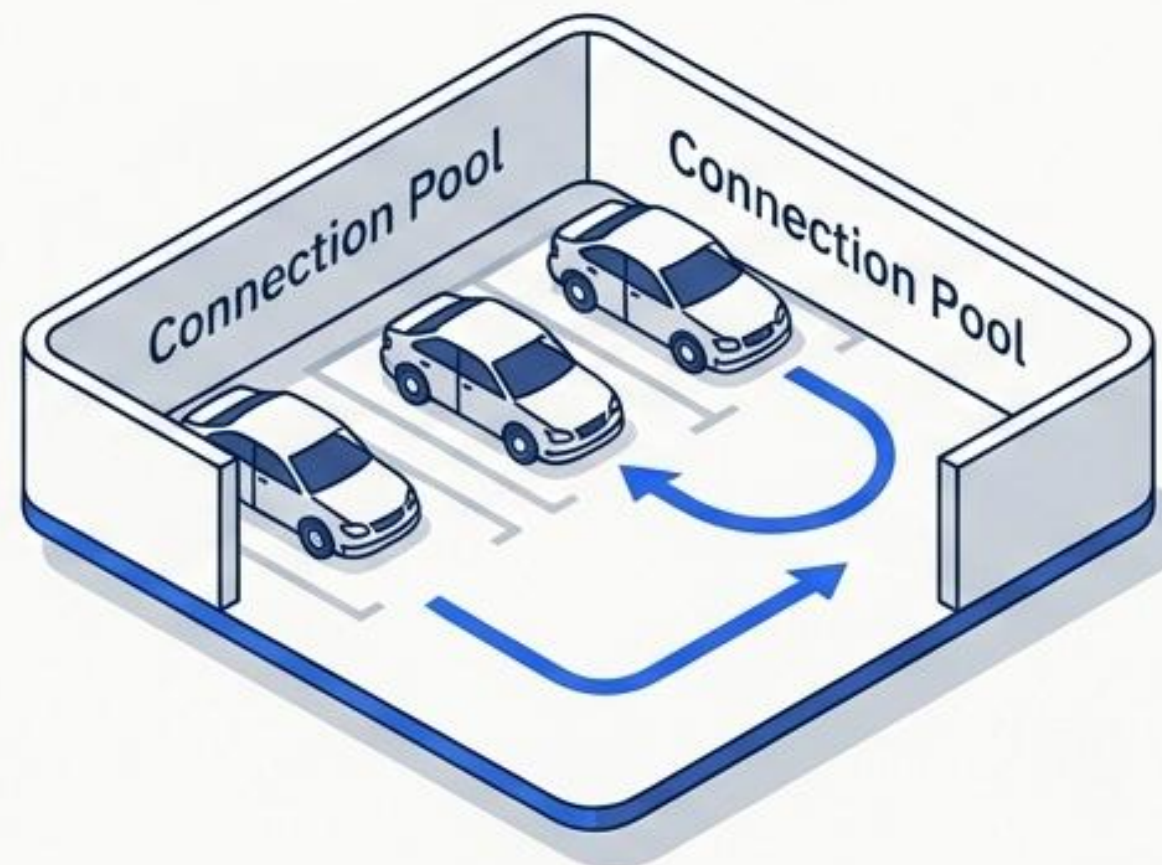
DBCP: 연결의 재사용과 트래픽 최적화

⚠ Without DBCP (JDBC만 사용)



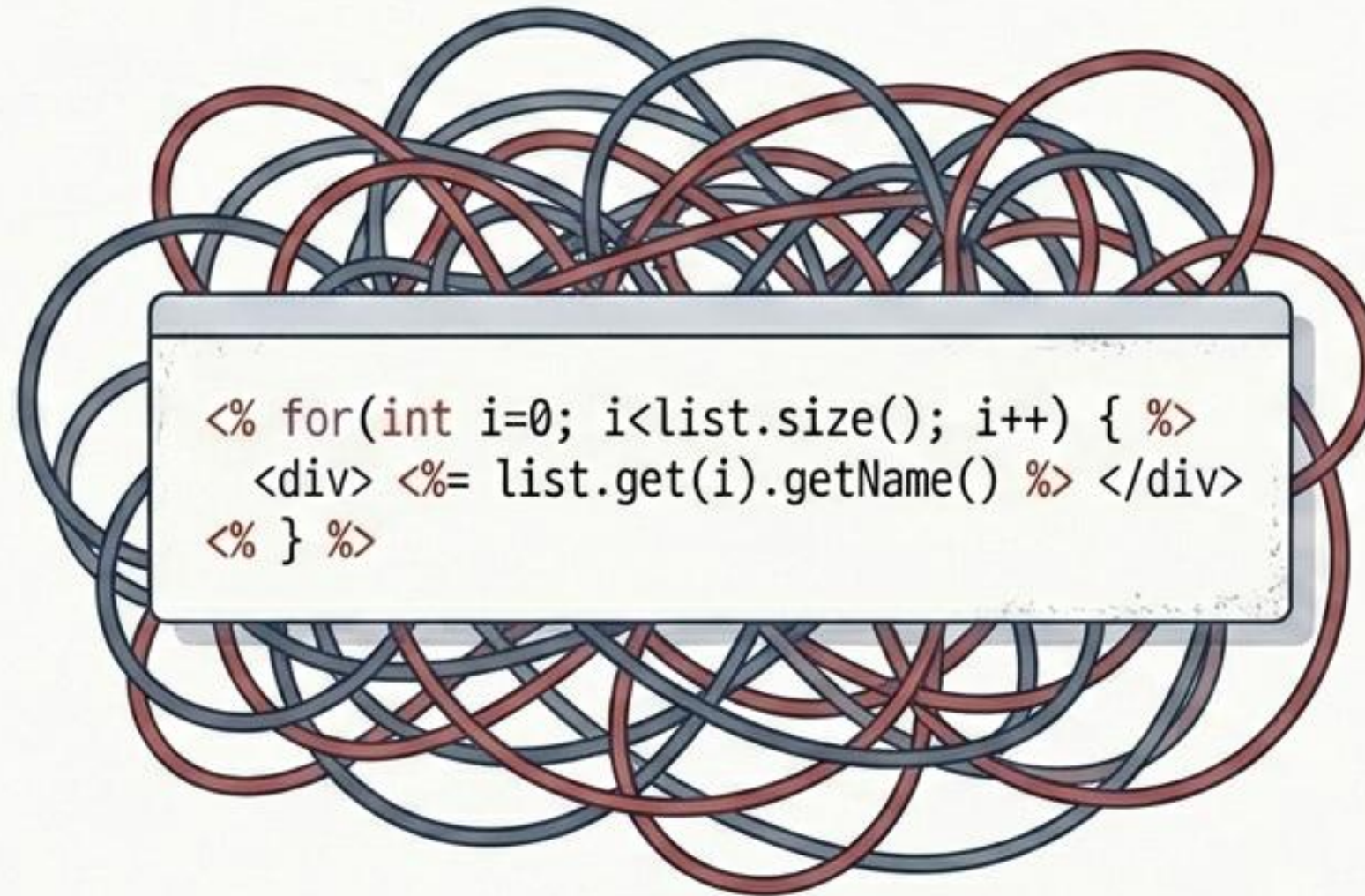
매번 새 자동차 구매.
접속 → 사용 → 종료.
서버 부하 극심, 매우 느림.

🔗 With DBCP (Connection Pool)



렌터카 회사에서 빌려 쓰기.
커넥션을 Pool에 미리 생성. 꺼내 쓰기 → 반납.
속도와 효율성 극대화 (실무 필수).

레거시 뷰(View)의 문제점: 스파게티 코드



HTML과 Java가 혼합되어 가독성이 극도로 떨어지고,
유지보수가 불가능에 가깝다.

EL & JSTL: HTML스러운 깔끔한 프레젠테이션

`<% = %>` (Legacy)



EL: Java 코드 없이
데이터 출력

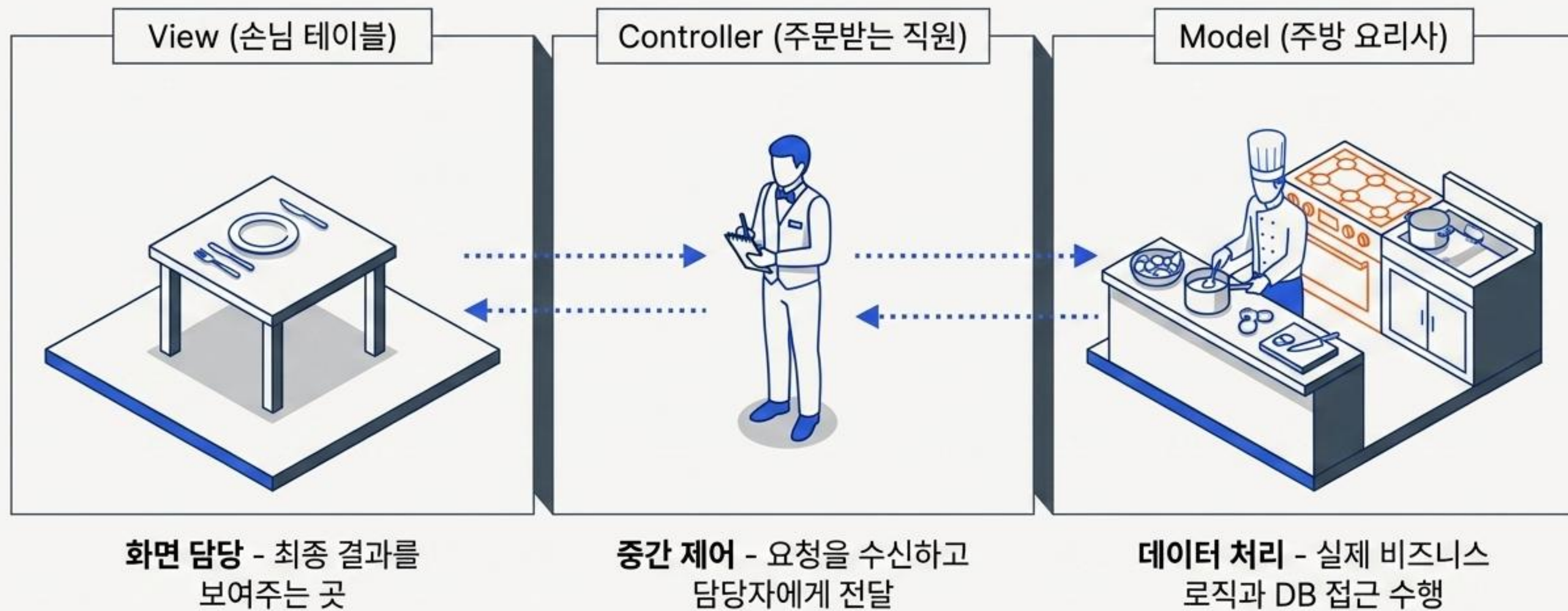
```
<c:forEach var="m" items="${list}">
    <div>${m.name}</div>
</c:forEach>
```

JSTL: 태그 기반의
반복문/조건문 처리

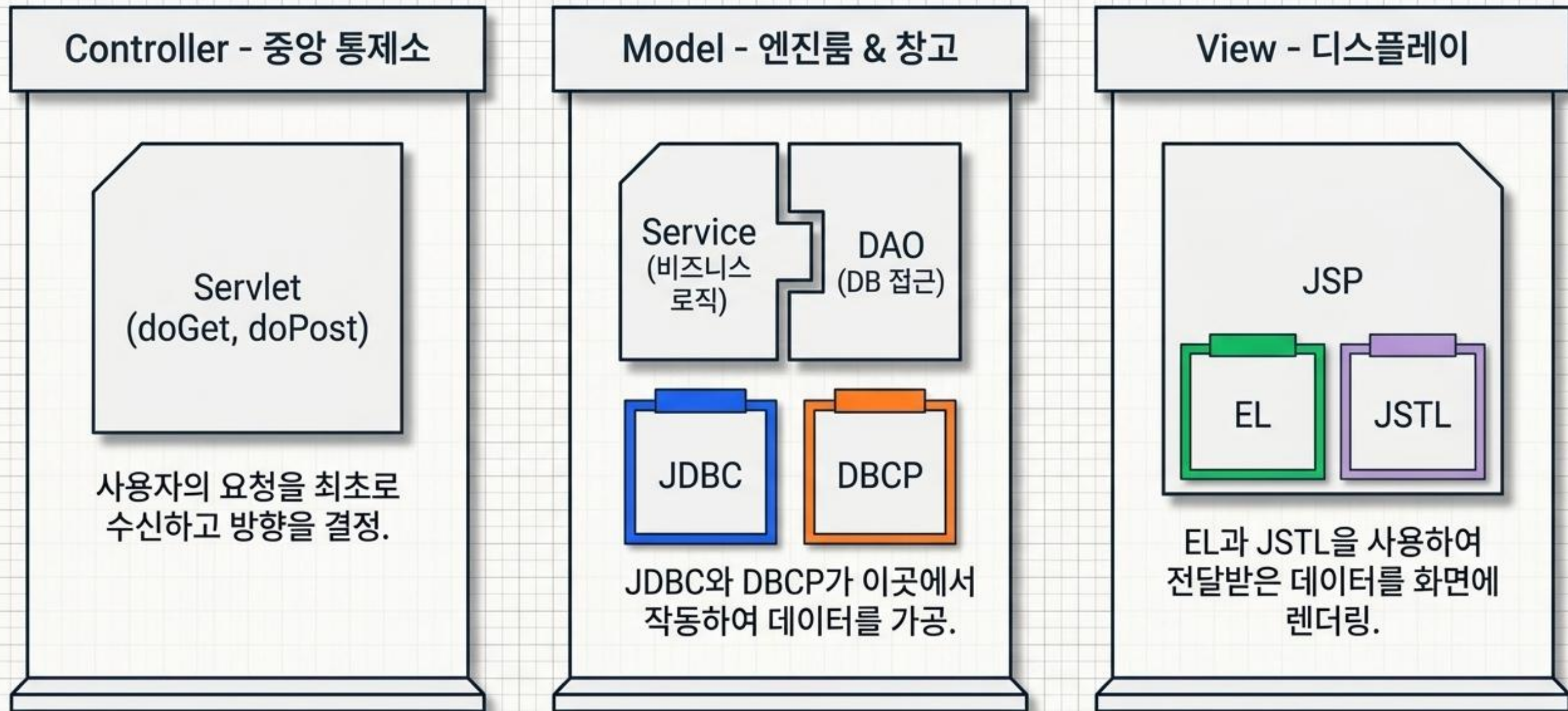
MVC 구조에 완벽하게 부합하는 직관적인 코드

MVC 아키텍처: 완벽한 역할 분리

유지보수성을 극대화하는 실무의 핵심 설계 패턴



시스템 물류망 배치도



실전 시뮬레이션: "회원 목록을 보여주세요"

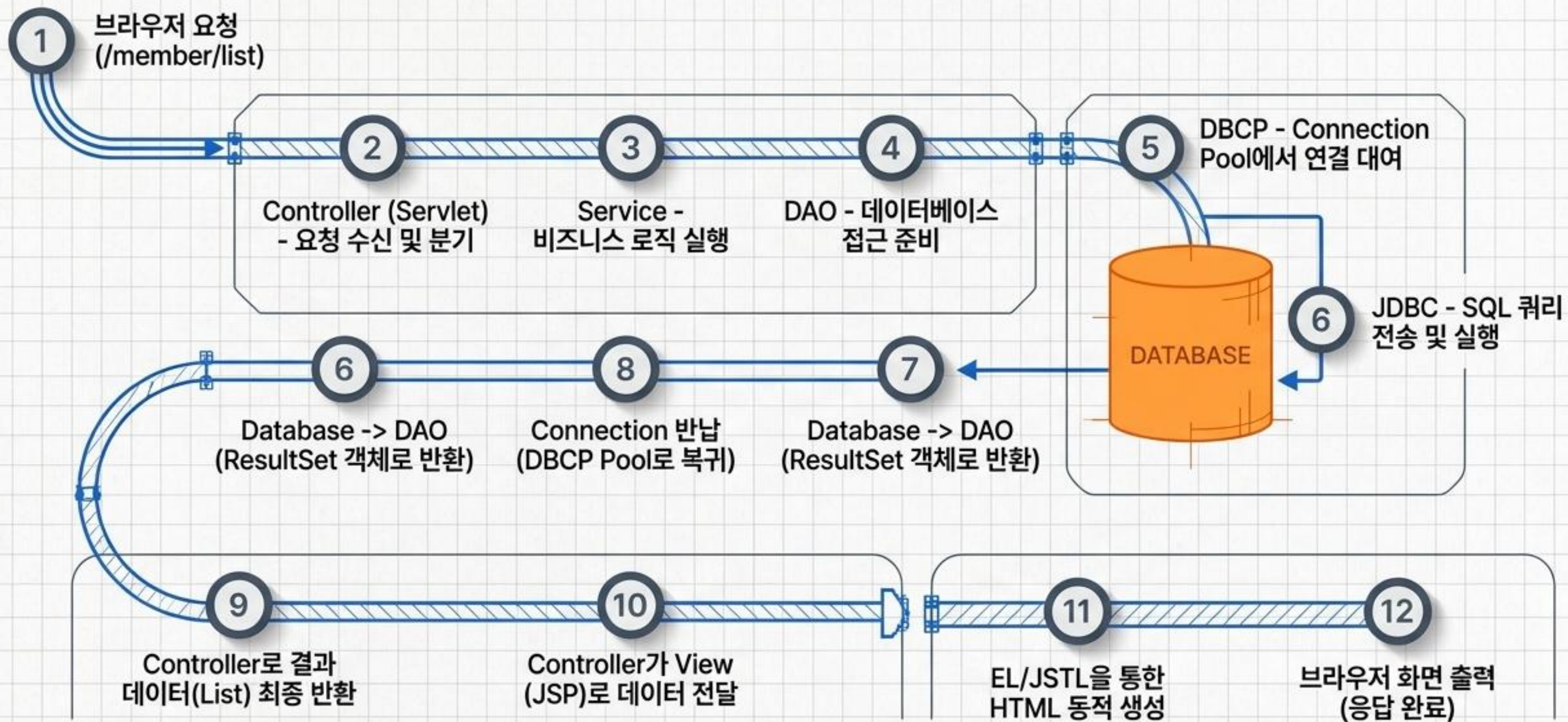


GET /member/list

사용자가 이 URL을 입력하는 순간,
백엔드 물류 센터에서는 어떤 일들이 순차적으로 일어날까요?

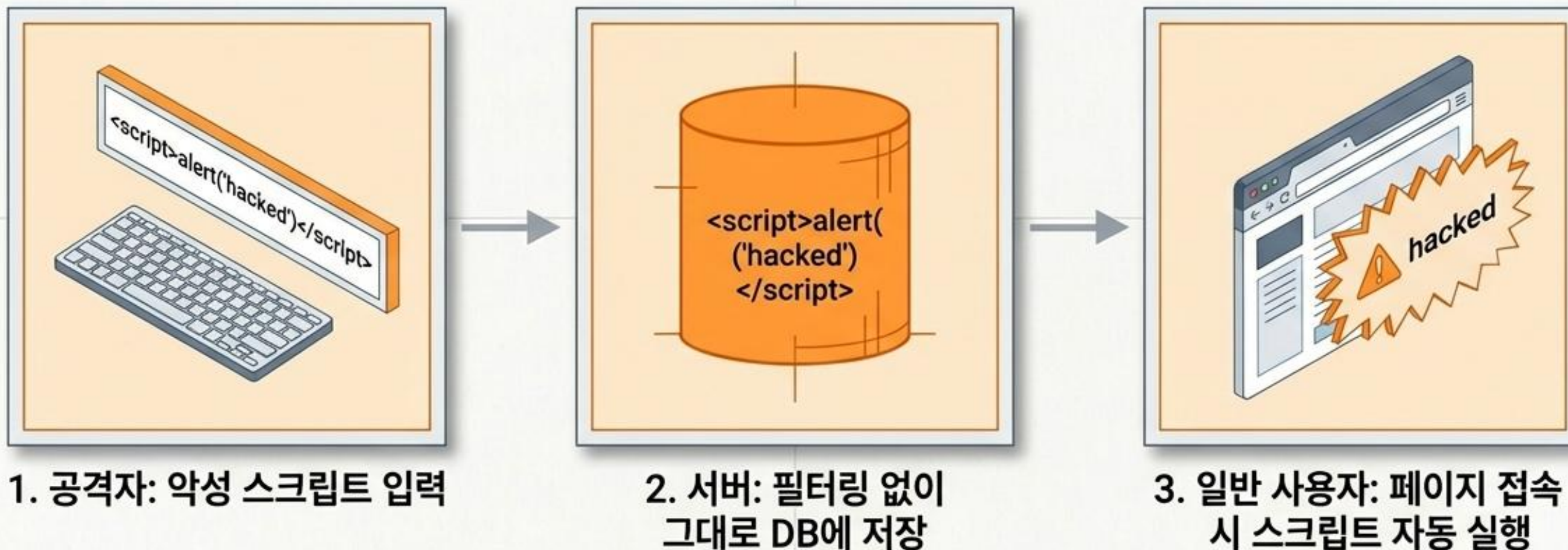
다음 슬라이드에서 12단계의 완벽한 데이터 흐름을 추적합니다.

웹 요청의 12단계 해부도



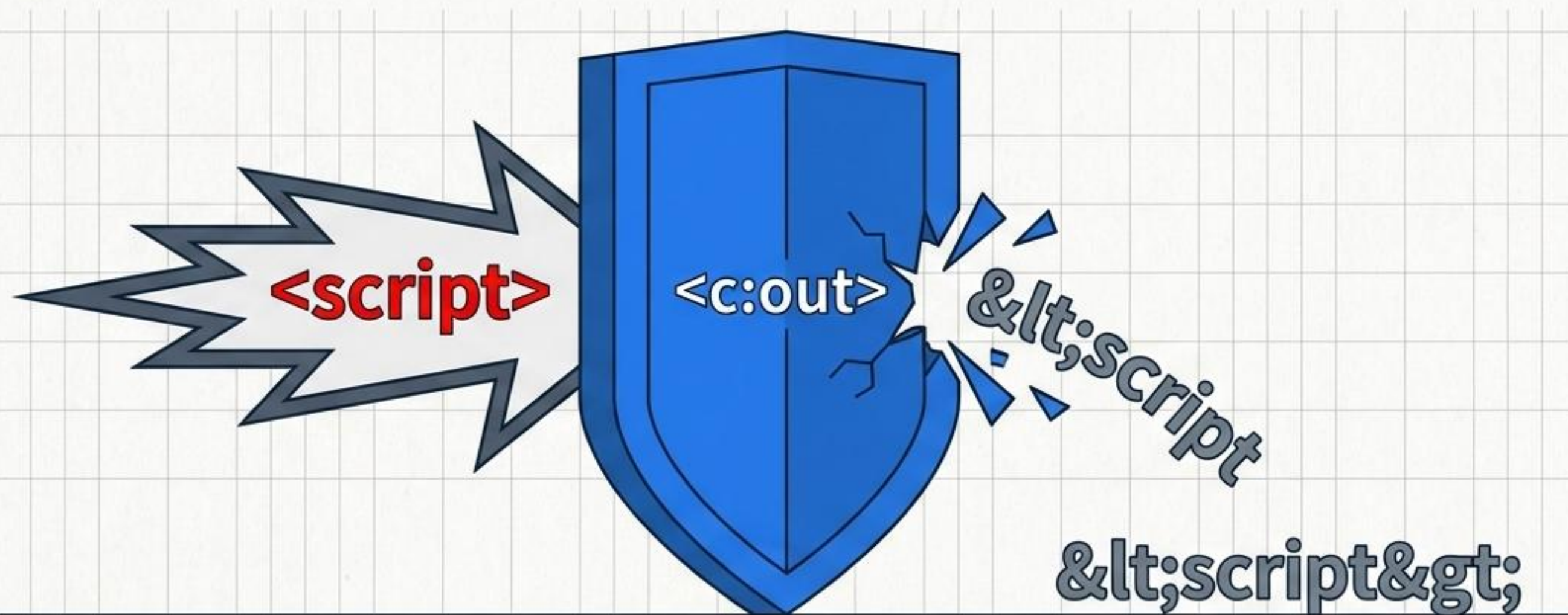
XSS (Cross-Site Scripting): 프론트엔드의 가장 큰 위협

단순한 텍스트 출력이 세션 탈취로 이어지는 과정



XSS 방어는 "출력 시점의 무력화(Escaping)"입니다.

<c:out>의 방어 매커니즘: 이스케이핑



JSTL의 <c:out> 태그는 HTML 특수문자를 자동으로 변환(Escape)하여 악성 스크립트가 브라우저에서 '실행'되지 않고 단순 '문자'로 보이게 만듭니다.

JSP 출력 방식 최종 평가 매트릭스

출력 방식	Java 의존도	XSS 보안성	가독성	MVC 적합성
<%= %> (스크립틀릿)	높음 (Bad)	❌ 위험 (방어 불가)	낮음 (Bad)	❌ 비적합 (레거시 전용)
\${ } (EL)	없음 (Good)	⚠️ 보통 (방어 없음)	높음 (Good)	✅ 적합 (기본 선택)
<c:out> (JSTL)	없음 (Good)	✅ 안전 (Escape 지원)	높음 (Good)	✅ 필수 (사용자 입력 출력 시)

면접을 위한 최종 무기 (Cheat Sheet)

핵심 암기장

JDBC: Java의 DB 접근 표준 API

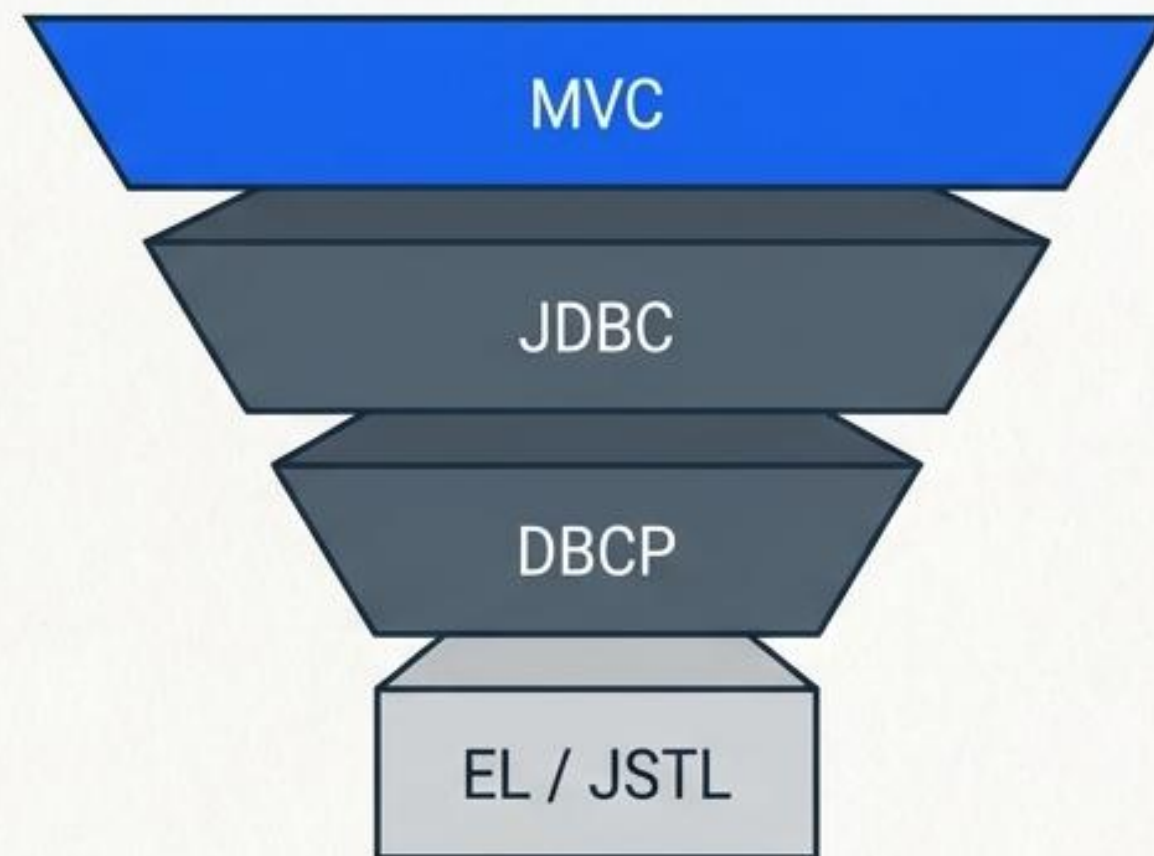
DBCP: Connection 생성 비용을 줄이는 Pool 기술

EL: JSP 값 출력 표현식

JSTL: JSP 태그 기반 로직 처리

MVC: 유지보수성을 위한 역할 분리 설계 패턴

실무 기준 중요도



핵심 주의: JDBC는 '실제 SQL 실행 기술'이며, DBCP는 'JDBC Connection 관리 기술'임을 명확히 구분할 것.

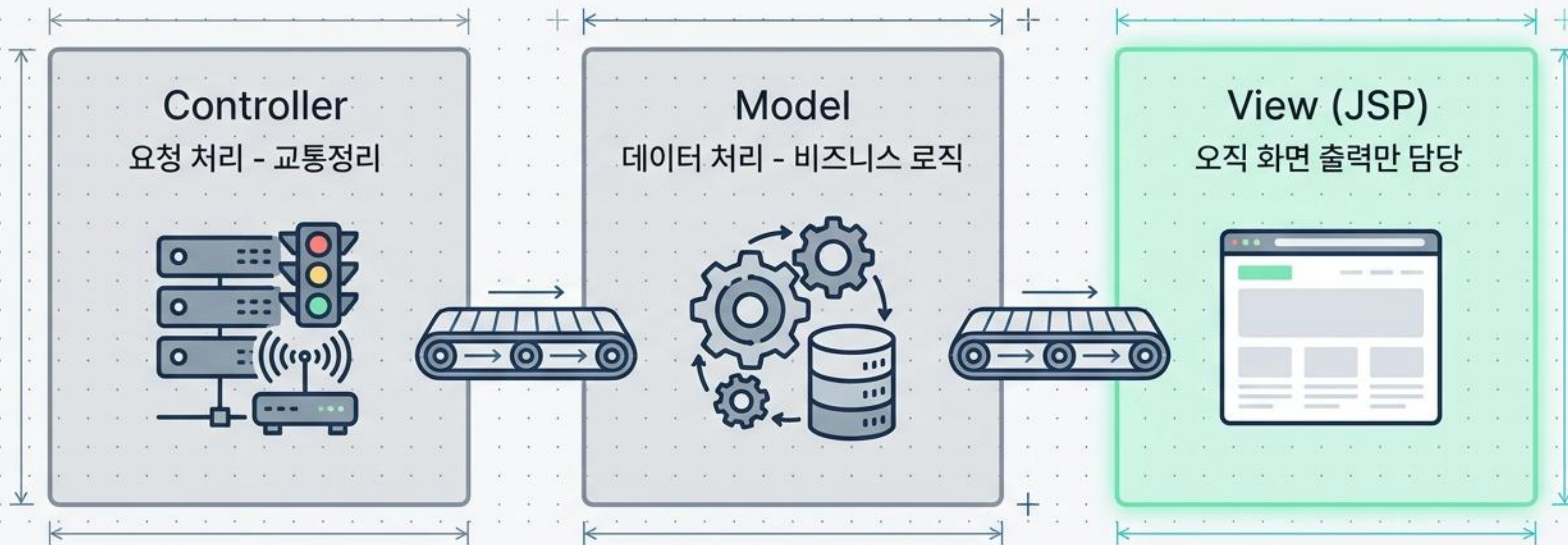
A terminal window with a light gray border and three colored window control buttons (red, yellow, green) in the top-left corner. The window contains a command prompt and a large title. A faint gear watermark is visible in the background of the terminal content.

```
> ./refactor_architecture.sh|
```

모던 웹 아키텍처의 완성: 레거시 코드 제거와 DB 병목 최적화

실무와 기술 면접을 관통하는 백엔드 코어 가이드

웹 아키텍처의 절대 원칙: 분리와 책임



JSP 내부에 비즈니스 로직(Java)이 침투하는 순간,
MVC의 근본 구조가 **붕괴**됩니다.

레거시 JSP의 참상: 스파게티 코드

```
<table>
```

```
<%for(Member m : list){%
```

```
<tr>
```

```
<%=m.getName()%></tr>
```

```
</tr>
```

```
<%}%>
```

```
</table>
```

Syntax Error

Syntax Error



? HTML인지 Java인지
? 시각적 구분 불가



View가 로직을 처리하는
아키텍처 위반



프론트엔드/디자이너와의
협업 불가능

실무에서 스크립틀릿(<% %>)을 혐오하는 5가지 이유



언어의 혼재

HTML과 Java의
무분별한 결합

유지보수 악몽

디자이너가 UI를
수정할 수 없는 구조

MVC 파괴

화면(View)이
데이터(Model)를
직접 조작

재사용 불가

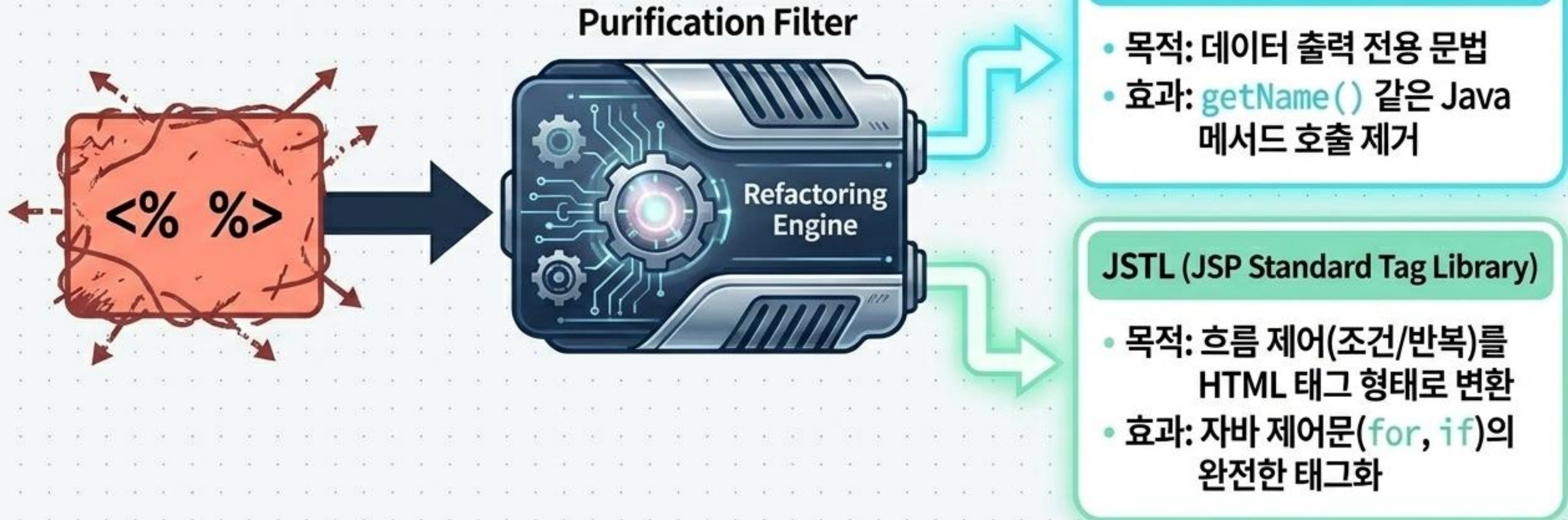
페이지마다
중복되는 로직
(Copy & Paste 지옥)

최악의 가독성

조건문과 반복문이
겹칠 때 발생하는
괄호 지옥

질서 회복: Java의 완전한 퇴출

JSP를 다시 순수한 화면(View)으로 되돌리기 위한 두 가지 무기



EL (Expression Language) 리팩토링 비교

기존 방식 (Legacy Scriptlet)

```
<%= member.getName() %>
```

- 코드가 불필요하게 김
- Java Getter 메서드 직접 호출
- HTML의 디자인 흐름을 끊음

모던 방식 (EL)

```
${member.name}
```

- 코드가 압도적으로 짧음
- Getter 메서드 자동 호출
- HTML/CSS와 완벽하게 어울리는 형태

EL 핵심 목적 = JSP 내부의 Java 연산 코드를 0%로 만드는 것

JSTL 리팩토링 비교: 제어문의 태그화

기존 방식 (Java for문)

```
<% for(Member m : list){ %>  
    <%= m.getName() %>  
<% } %>
```

- 괄호와 태그가 엉켜 디자이너가 읽을 수 없음

모던 방식 (JSTL)

```
<c:forEach var="m" items="${list}">  
    ${m.name}  
</c:forEach>
```



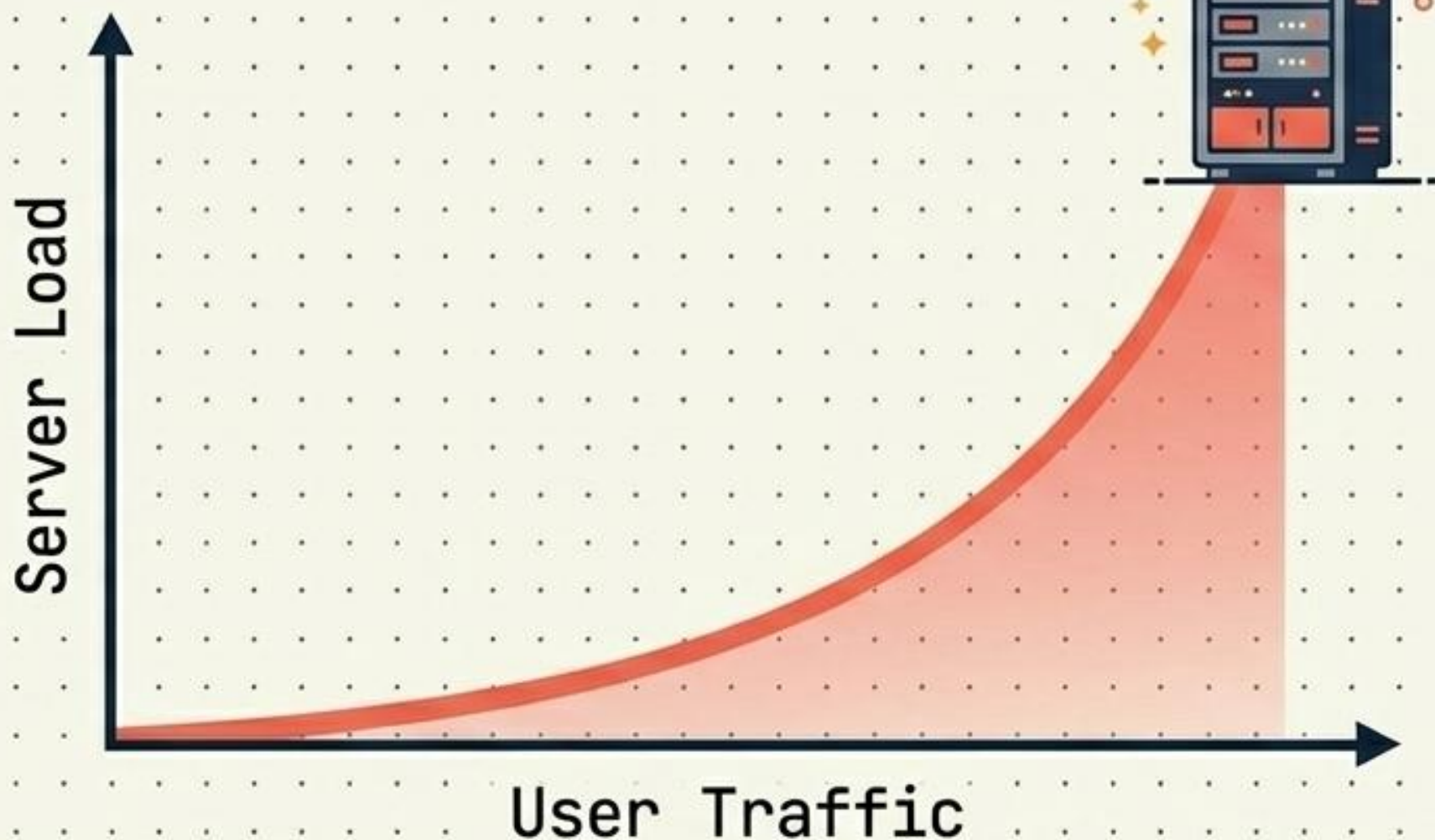
직관적인 HTML 태그 형태. 디자이너와 프론트엔드 개발자가 거부감 없이 UI를 유지보수할 수 있는 완벽한 뷰(View) 템플릿 완성.

[실무 매트릭스] MVC 기준 JSP 기술 우선순위

가장 중요 (Core)		→ EL (`\${}`) & JSTL (<c:...>): 화면 출력과 제어의 표준. → 내장 객체 (`request`, `session`): Controller가 넘겨준 데이터와 로그인 정보 수신.
기본 필수 (Basic)		→ page 지시자: 한글 깨짐 방지 및 페이지 설정 → include 지시자: 헤더/푸터 등 공통 UI 컴포넌트 재사용
사용 금지 (Banned)		→ 스크립틀릿 (`<% %>`) & 표현식 (`<%= %>`): 아키텍처 붕괴의 주범. 즉시 제거 대상.

백엔드의 위기: 1,000명의 유저, 1,000번의 연결

1. 순수 JDBC는 매 요청마다 Connection 생성
→ SQL 실행 →
→ Connection 종료를 반복.
2. 동시 접속자가 1,000명이 되면,
서버는 1,000번의 Connection
을 동시에 생성하려 시도.



메모리 고갈



DB 서버 과부하



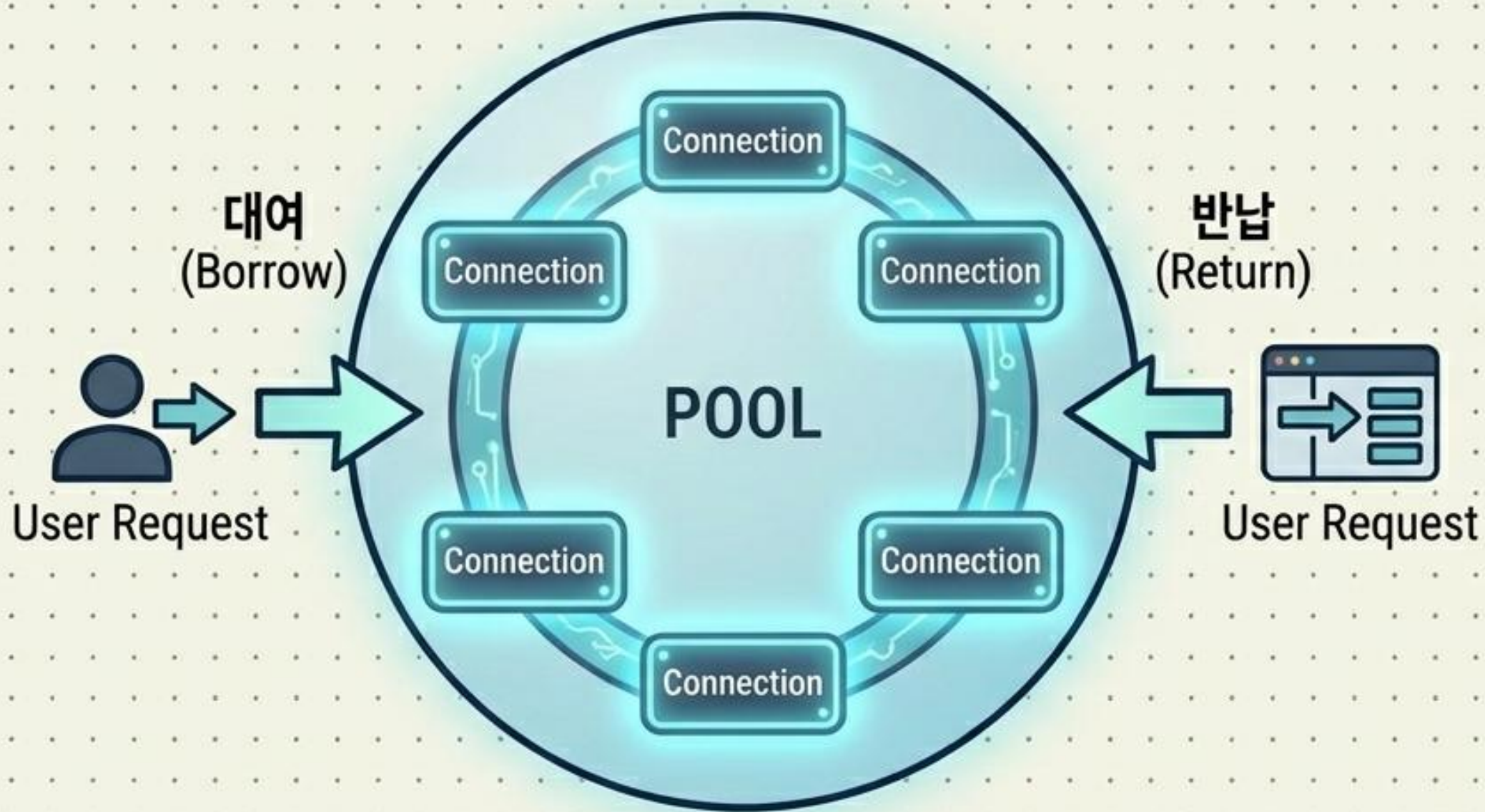
응답 속도
기하급수적 저하

DB 연결의 숨겨진 비용



DB 연결은 단순한 Java 객체 생성이 아닙니다.
네트워크와 서버의 물리적 자원을 극도로 소모하는 가장 무거운 작업입니다.

성능의 마법: DBCP (Database Connection Pool)



[사전 준비]

서버가 켜질 때 Connection을 미리 여러 개 생성하여 Pool에 저장.

[대여]

사용자 요청 시, 새로 만들지 않고 Pool에서 즉시 대여.

[반납]

SQL 실행이 끝나면 연결을 파괴하지 않고 다시 Pool로 반납 (재사용).

결과: 가장 무거운 작업(연결 생성)의 비용이 '0'에 수렴. 압도적인 응답 속도 향상.

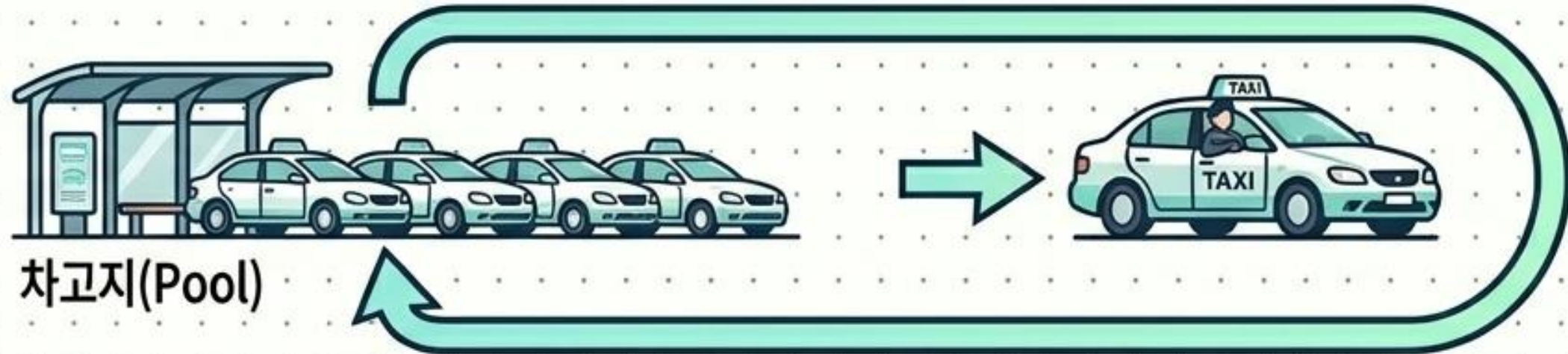
[개념 비유] 택시를 타는 두 가지 방법

JDBC 방식



택시를 탈 때마다 공장에서 새 차를 조립하고, 내릴 때 차를 폐차시킴. (극단적 시간 낭비와 비효율)

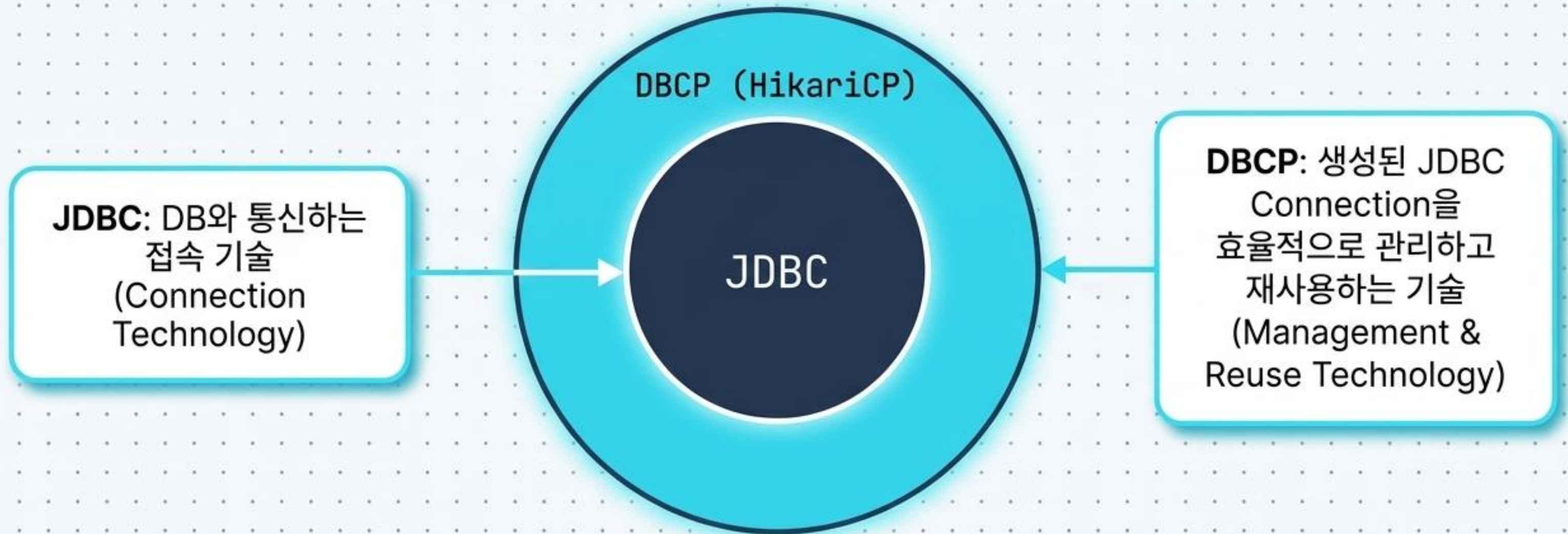
DBCP 방식



차고지(Pool)에서 이미 대기 중인 차를 타고 이동 후 목적지에 도착하면 차고지로 반납. (초고속, 극대화된 자원 효율)

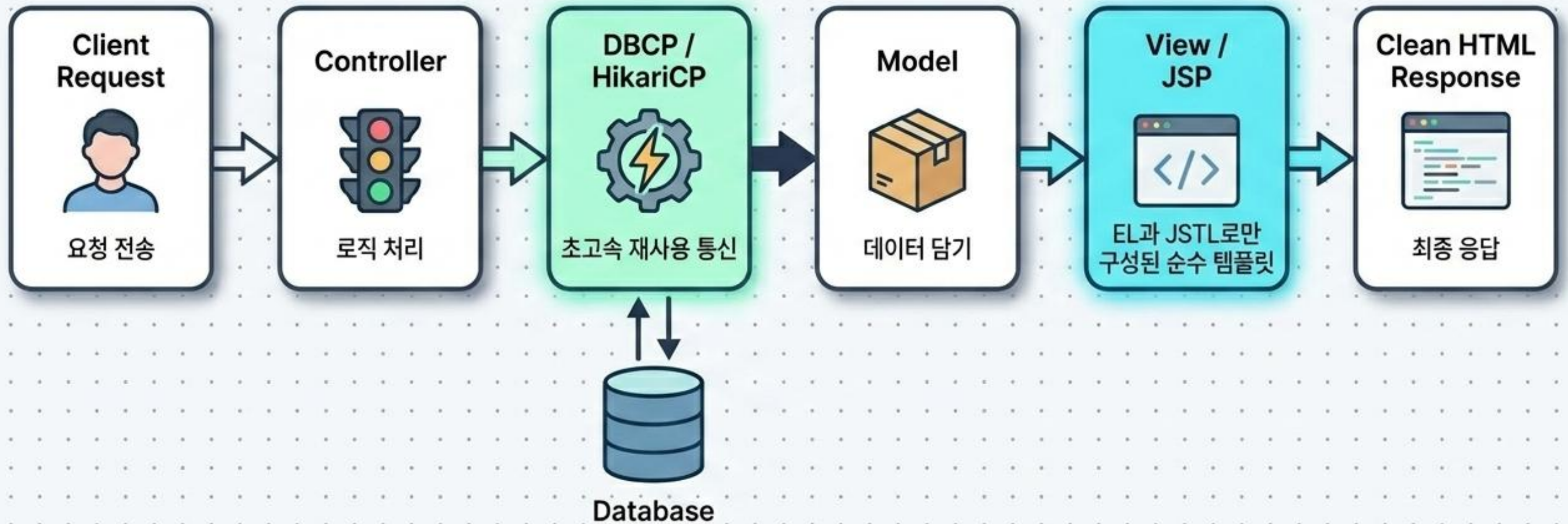
아키텍처 계층 정리: DBCP vs JDBC

DBCP는 JDBC를 대체하는 기술이 아닙니다.



현재 실무 및 Spring Boot의 절대적 기본 표준(Default)은 HikariCP입니다.

모던 MVC 아키텍처 마스터 다이어그램



백엔드는 자원을 극도로 최적화하고, 프론트엔드는 역할의 분리를 완벽히 이뤄낸 실무 표준 아키텍처.

기술 면접 핵심 치트시트

View Refactoring

Q: JSP에서 스크립틀릿(<% %>)을 제거해야 하는 이유는?

A: HTML과 Java가 혼재되어 유지보수성이 떨어지며, 화면 출력만 담당해야 할 View가 로직을 처리하게 되어 MVC 역할 분리의 원칙이 깨지기 때문입니다.

EL & JSTL

Q: EL과 JSTL의 도입 목적은?

A: JSP에서 Java 연산 코드를 완전히 배제하고, 데이터를 직관적으로 출력(EL)하며 제어문을 태그화(JSTL)하여 완벽한 View 템플릿을 구축하기 위함입니다.

Database Optimization

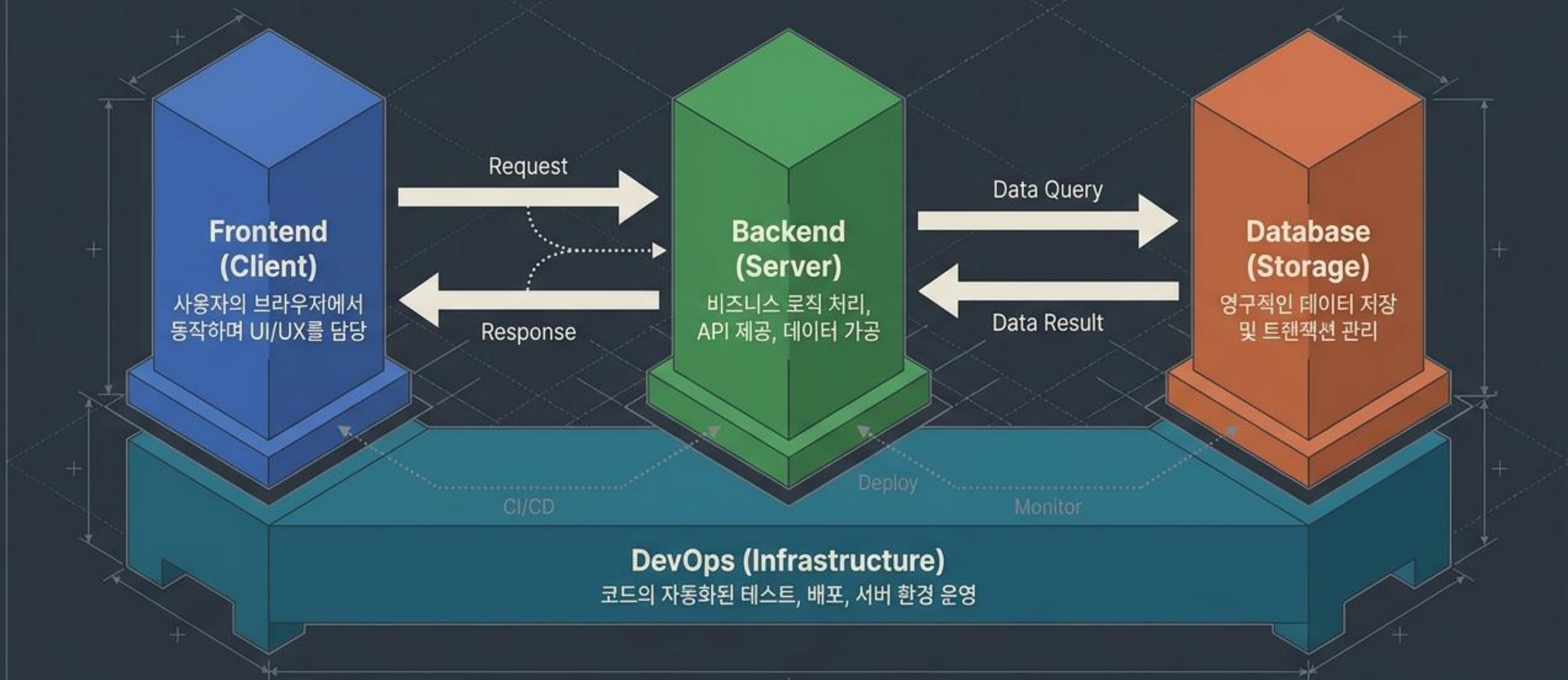
Q: 순수 JDBC 대신 DBCP를 사용하는 이유는?

A: 네트워크 통신 등 비용이 매우 큰 Connection 생성 작업을 제거하기 위해, 미리 연결을 생성해두고 재사용하여 서버 과부하를 막고 응답 성능을 극대화하기 위함입니다.

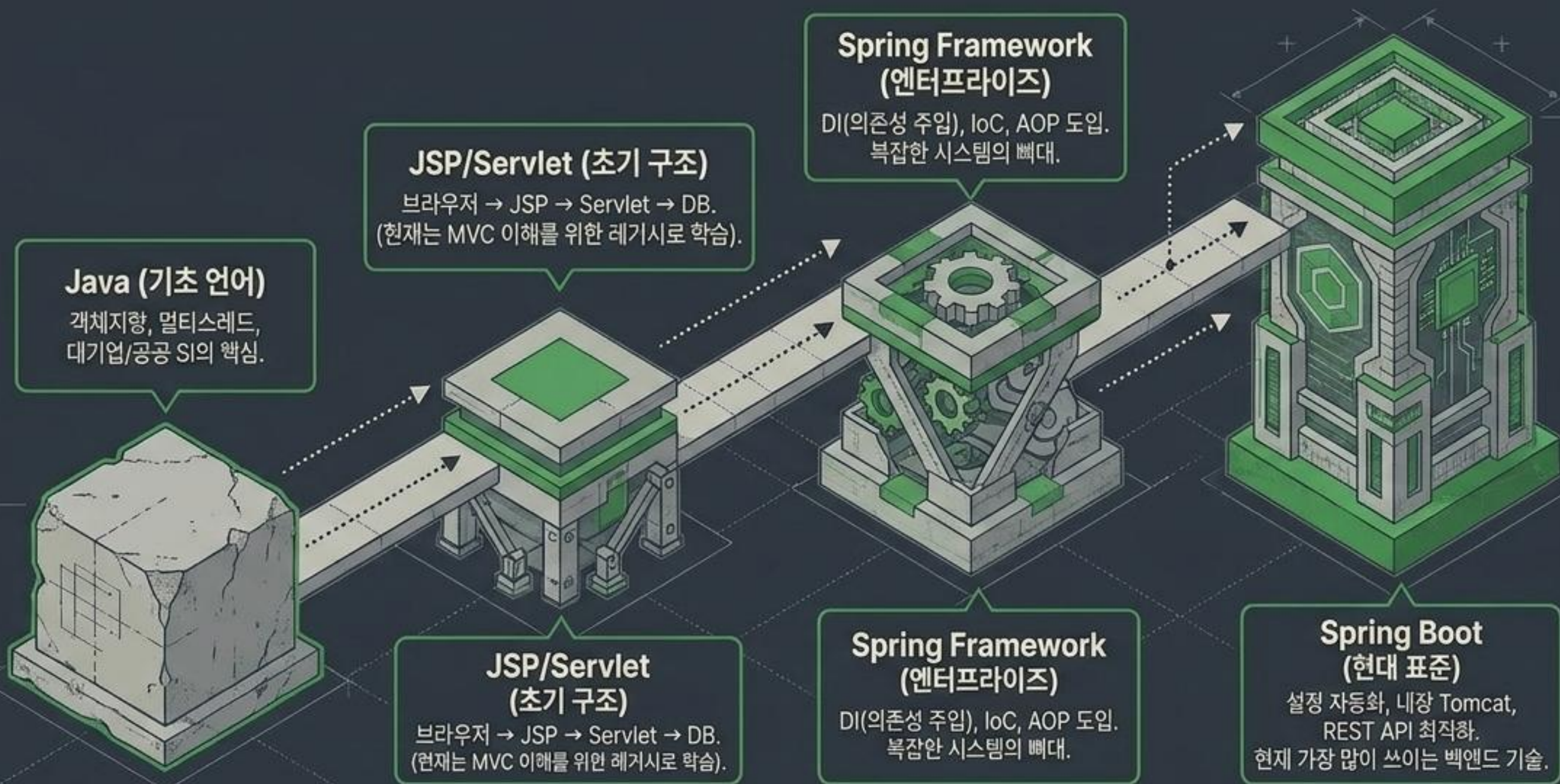
2026 풀스택 웹 개발 마스터 청사진

Java/Spring 기반 백엔드부터 모던 프론트엔드, DevOps 인프라까지

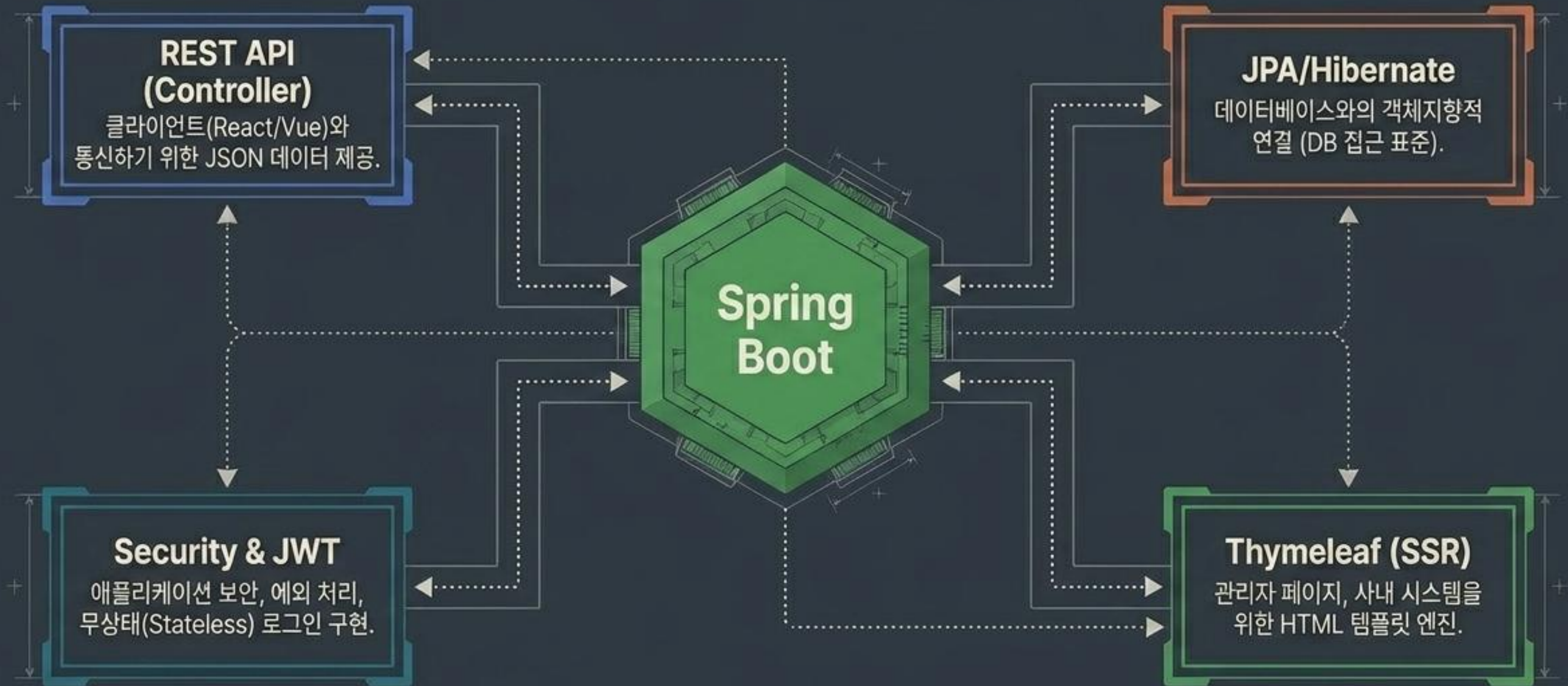
현대 웹 애플리케이션을 지탱하는 4개의 기둥



백엔드의 진화: 기초 언어에서 사실상의 표준 프레임워크로



모던 백엔드의 중심 허브, Spring Boot



데이터베이스 연결 계층의 추상화 과정

MyBatis / JPA

개발자가 실제로 다루는 추상화된
데이터 접근 인터페이스.

DBCP / HikariCP (Connection Pool)

DB 연결을 미리 여러 개 생성해두고 빌려쓰는
풀 관리. 성능 저하 문제 해결.

Connection Pool

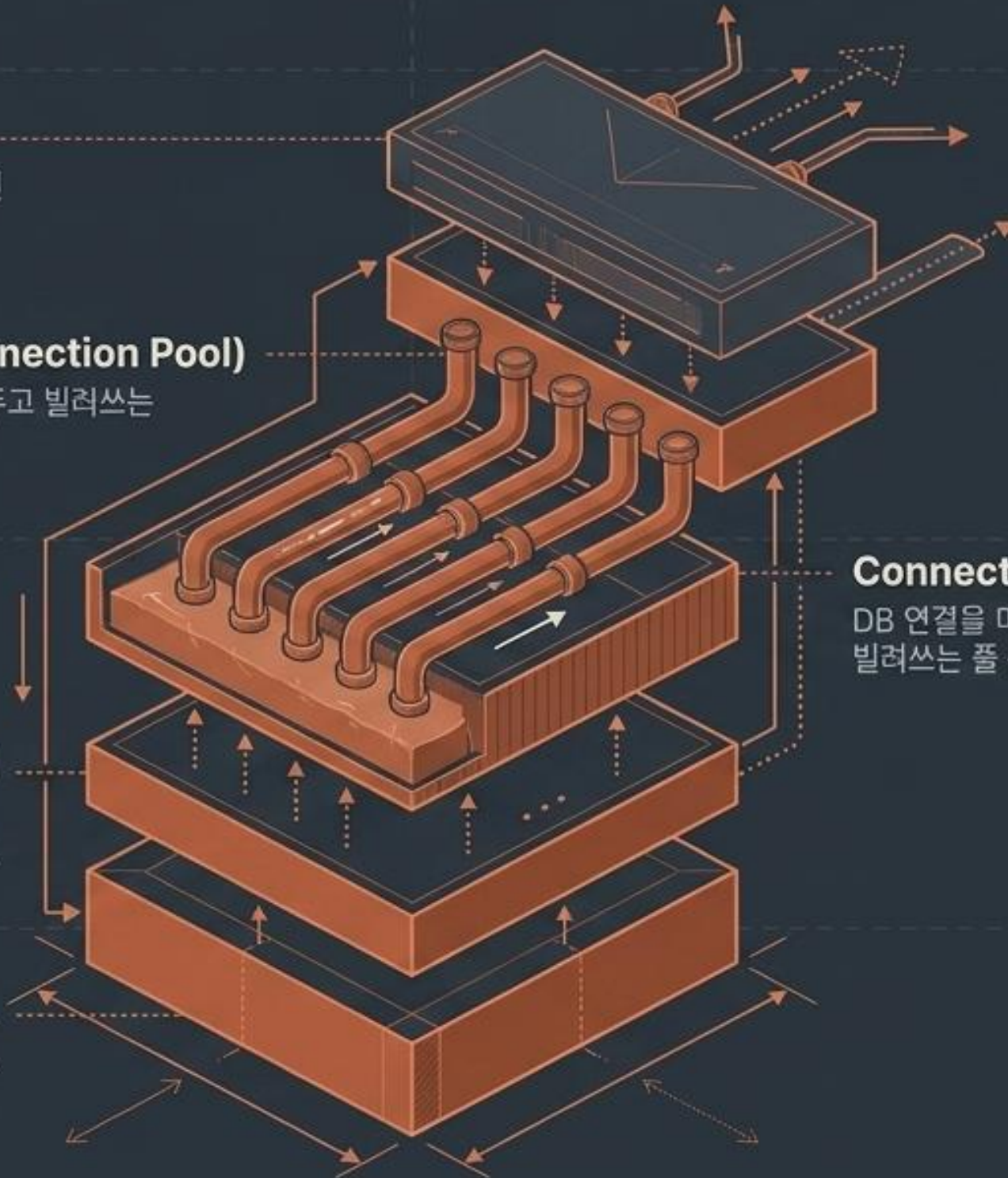
DB 연결을 미리 여러 개 생성해두고
빌려쓰는 풀 관리.

JDBC (기본 API)

Java가 DB에 접속하는 뼈대 기술.
연결-실행-종료 반복으로 코드량 증가.

Oracle / MySQL / PostgreSQL

실제 데이터가 저장되는 RDBMS (Storage).



데이터 영속성 패러다임: MyBatis vs JPA

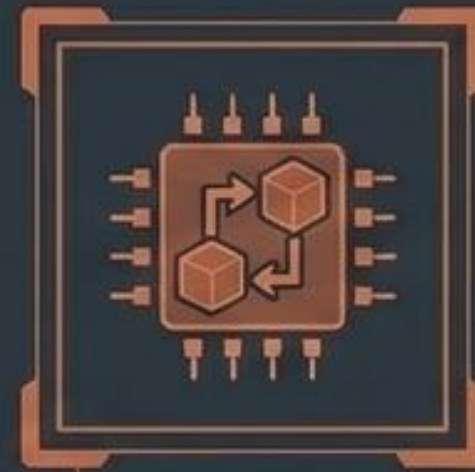


MyBatis (SQL 중심 매퍼)

- ▶ **방식**: 개발자가 SQL을 직접 작성
(`<select id=...>`)

```
SELECT * from list/  
FROM "list" + "list" + "list", 'neopen_casett...');
```

- ▶ **장점**: SQL 제어가 매우 강함,
복잡한 통계 쿼리 및 튜닝 용이.
- ▶ **단점**: CRUD 반복 작성 필요,
유지보수 시 SQL 코드량 증가.
- ▶ **사용처**: 공공기관, 금융권, 전통적인 SI
(Oracle 중심).



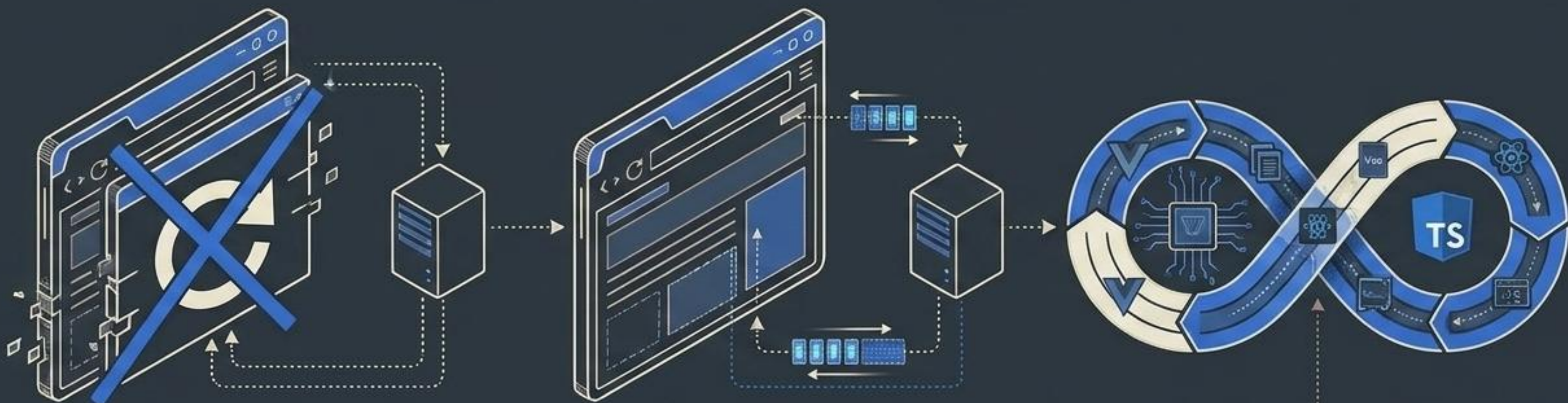
JPA (객체 중심 ORM)

- ▶ **방식**: 객체와 DB 테이블을 자동 매핑
(`userRepository.save(user)`)

```
public (userRepository.save(user))
```

- ▶ **장점**: 압도적인 생산성, 객체지향적 개발,
Spring Boot 생태계 표준.
- ▶ **단점**: 복잡한 쿼리 작성 불편(QueryDSL 필요),
성능(N+1) 러닝커브.
- ▶ **사용처**: 스타트업, 현대 웹 서비스
(MySQL/PostgreSQL 중심).

프론트엔드 패러다임의 변화: 새로고침 없는 웹의 탄생



과거 (정적 렌더링)

HTML/CSS/JS → jQuery →
요청 시마다 페이지 전체 새로고침 발생.

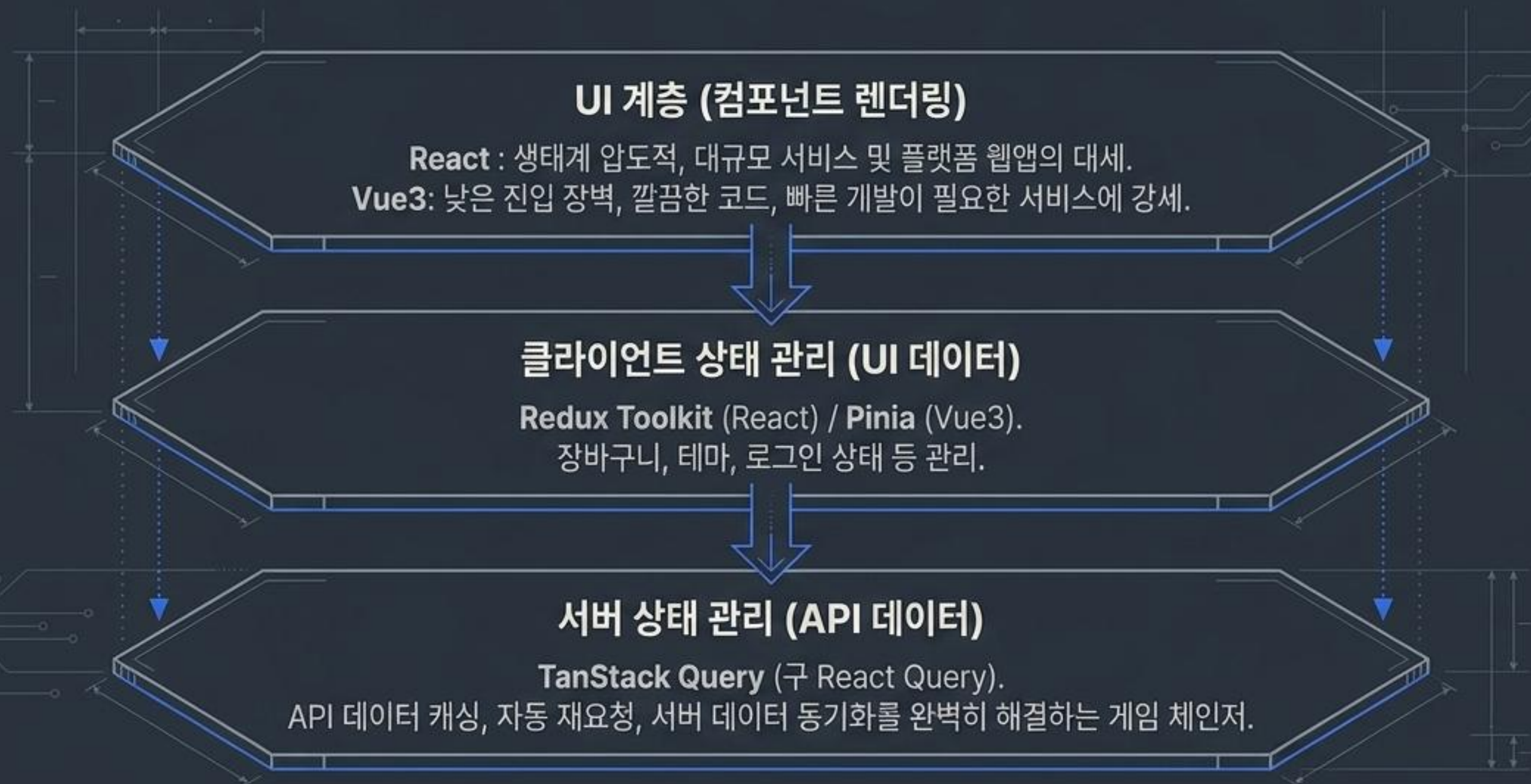
과도기 (비동기 통신)

AJAX 등장.
페이지 전체를 바꾸지 않고
서버와 데이터만 교환 (fetch).

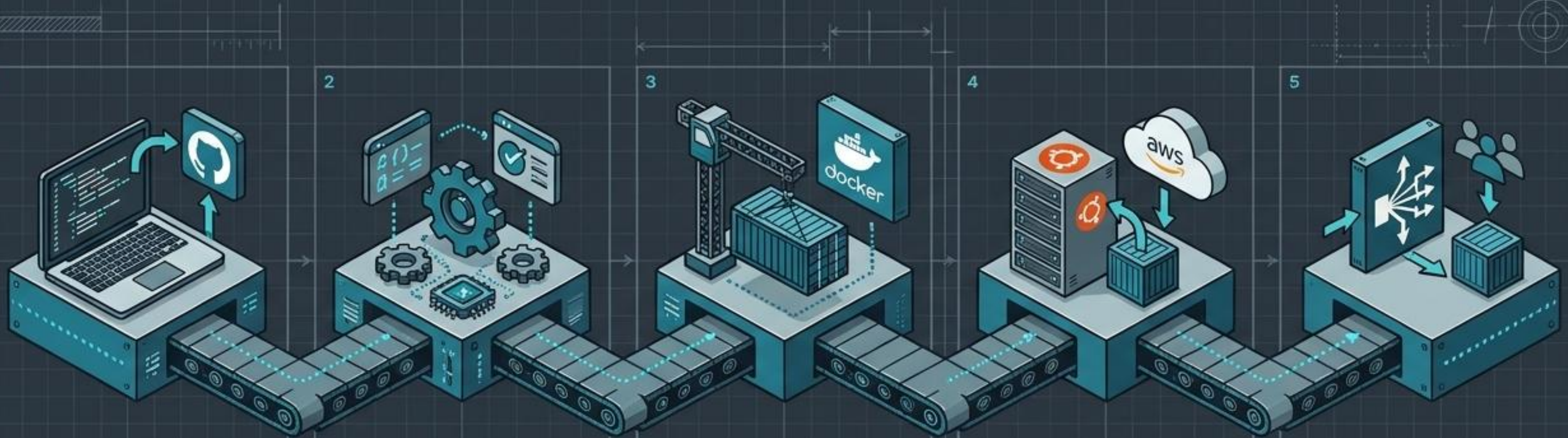
현재 (SPA 시대)

Vue/React 프레임워크 도입 →
상태 기반의 컴포넌트 렌더링
→ TypeScript를 통한 타입 안전성 확보.

모던 프론트엔드 애플리케이션의 3단 구조



인프라 & DevOps 라이프사이클: 코드가 서비스가 되기까지



Step 1: GitHub Push

개발자의 코드 작성 및
저장소 푸시.

Step 2: GitHub Actions

CI/CD 파이프라인을 통한
자동 테스트 및 빌드.

Step 3: Docker Hub

빌드된 애플리케이션을
컨테이너(이미지)로 묶어 저장.

Step 4: Ubuntu Server

AWS EC2 등 실제 서버 환경에
서 최신 이미지 Pull 및 실행.

Step 5: Nginx

실행된 컨테이너로 들어오는
사용자 트래픽 라우팅.

컨테이너와 배포 도구의 목적별 매트릭스

컨테이너 실행 및 관리

- **Docker**: 단일 애플리케이션 컨테이너화.
"내 PC에선 되는데 서버에선 안 됨" 문제 해결.
- **Docker Compose**: Spring Boot + DB + Redis 등 여러 컨테이너를 로컬에서 한 번에 띄우고 테스트.
- **Kubernetes (k8s)**: 수백 개의 컨테이너, 자동 복구, 오토스케일링을 관리하는 대기업/MSA 인프라 오케스트레이션.



CI/CD (지속적 통합/배포)

- **GitHub Actions**: GitHub 내장, 설정 간편, 스타트업 대세.
- **Jenkins**: 서버 직접 구축, 강력한 플러그인, 금융/대기업 및 SI 표준.



트래픽의 수문장: Nginx와 네트워크 라우팅

Nginx의 4대 핵심 역할



1. 리버스 프록시: 80(HTTP) 포트 요청을 안전하게 내부 8080 포트로 전달.



2. 정적 파일 처리: HTML, CSS, 이미지를 백엔드를 거치지 않고 빠르게 제공.



3. 로드밸런싱: 트래픽을 여러 서버로 분산.



4. HTTPS 설정: SSL 인증서 처리.

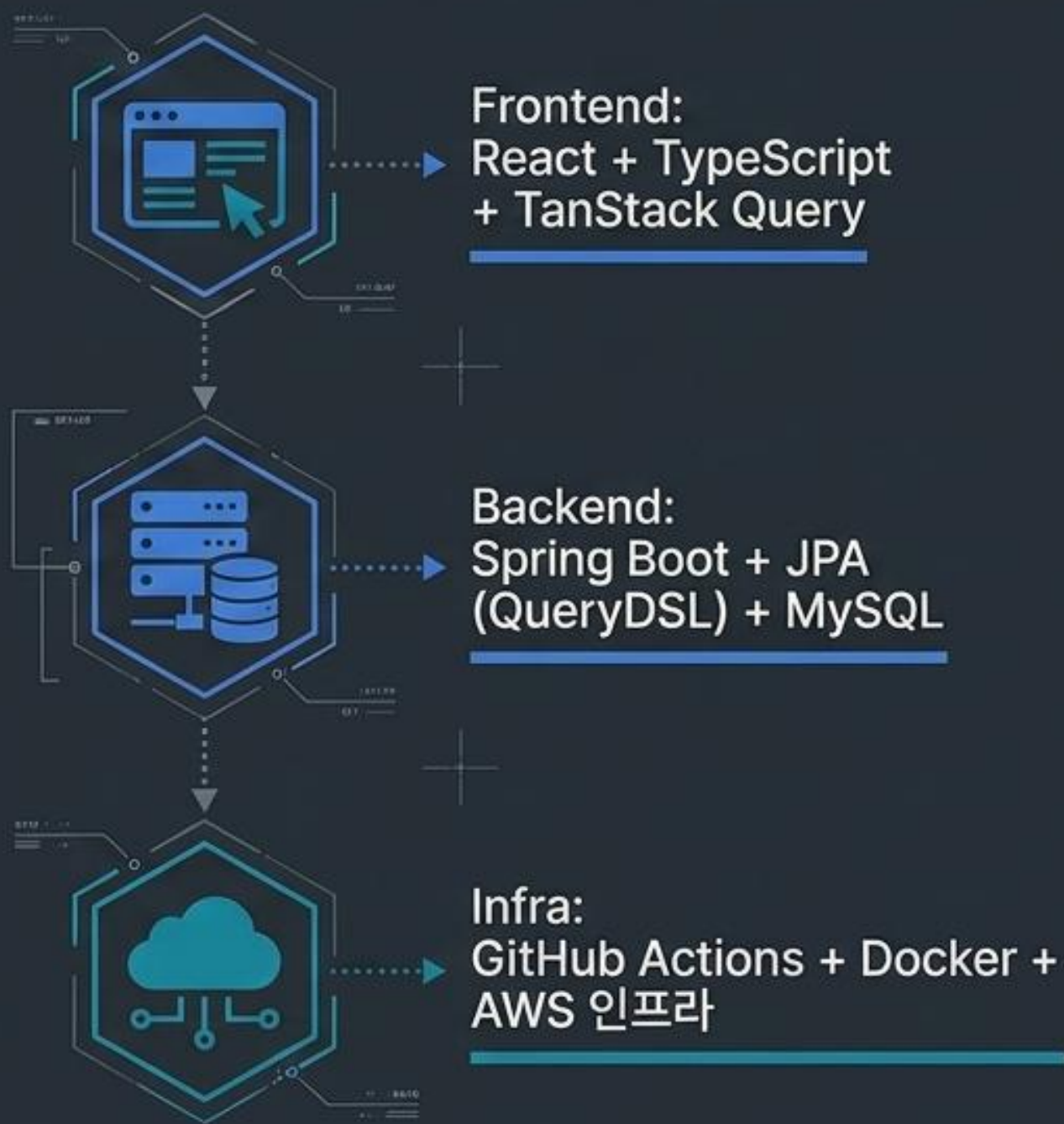


ngrok (로컬 외부 공개)

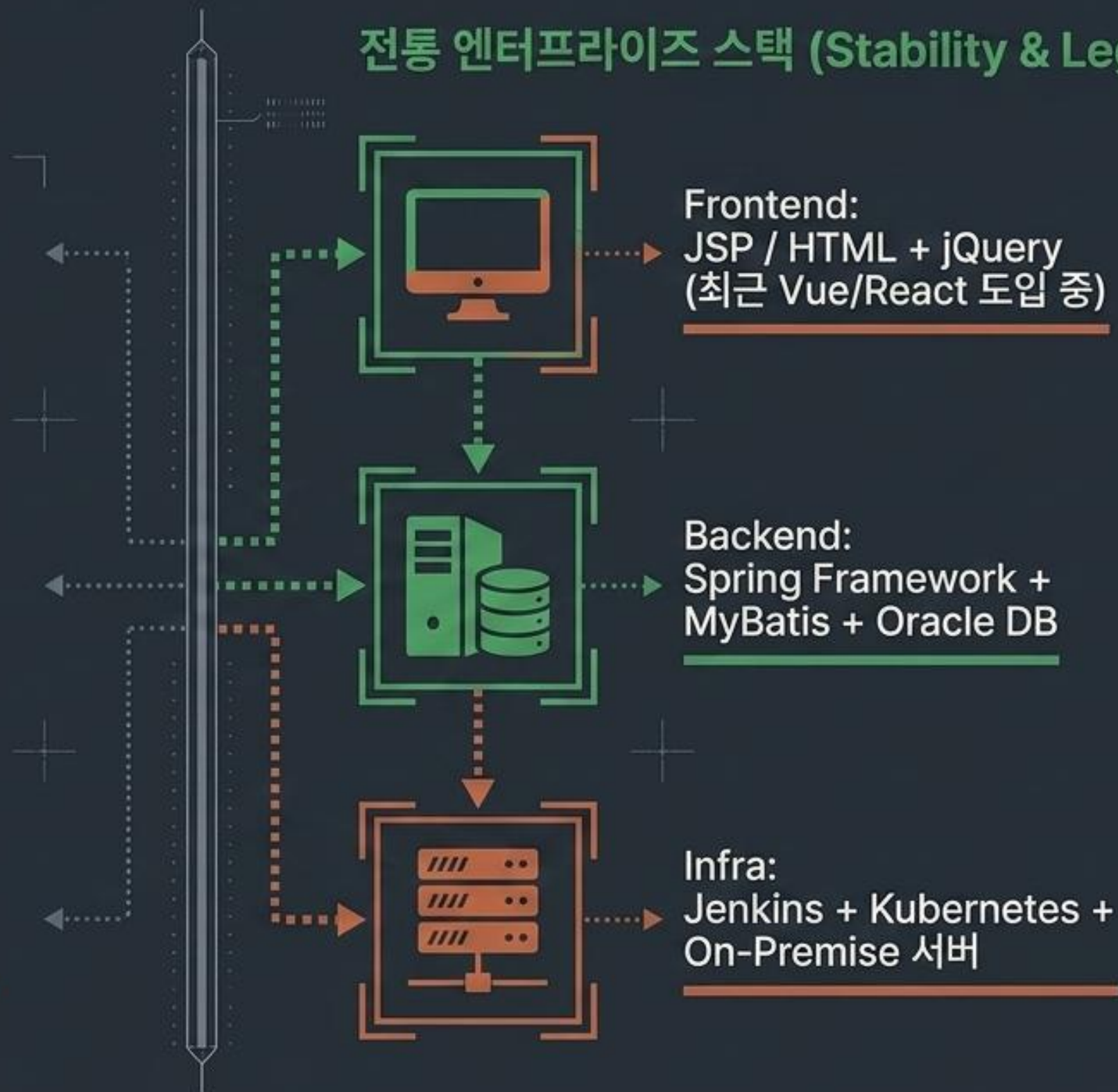
로컬 호스트(localhost:8080)를 외부 인터넷 URL로 연결.
웹훅(Webhook) 및 외부 API 연동 테스트 시 필수.

실무 기술 스택 종합: 두 가지 산업 경로

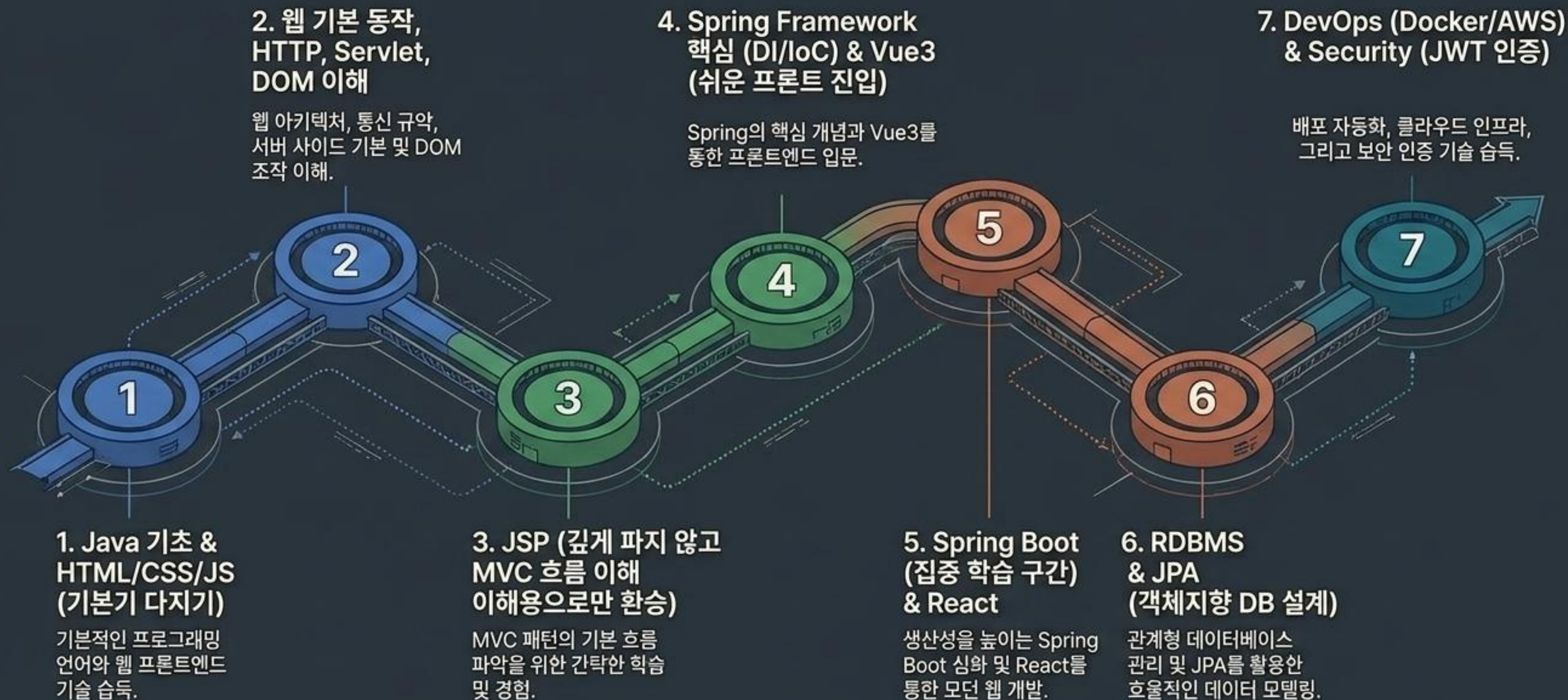
모던 스타트업 스택 (Agile & Scale)



전통 엔터프라이즈 스택 (Stability & Legacy)



2026 풀스택 마스터를 위한 권장 학습 노선도



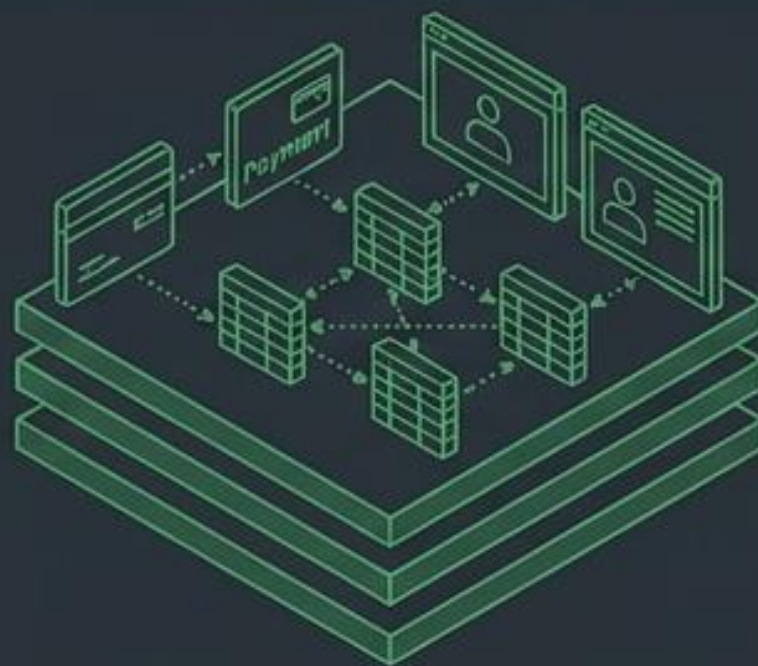
포트폴리오 마일스톤: 단계별 프로젝트 가이드



Beginner (게시판 / To-Do)

목표: 기본 CRUD, 세션 기반 로그인, DB 연동.

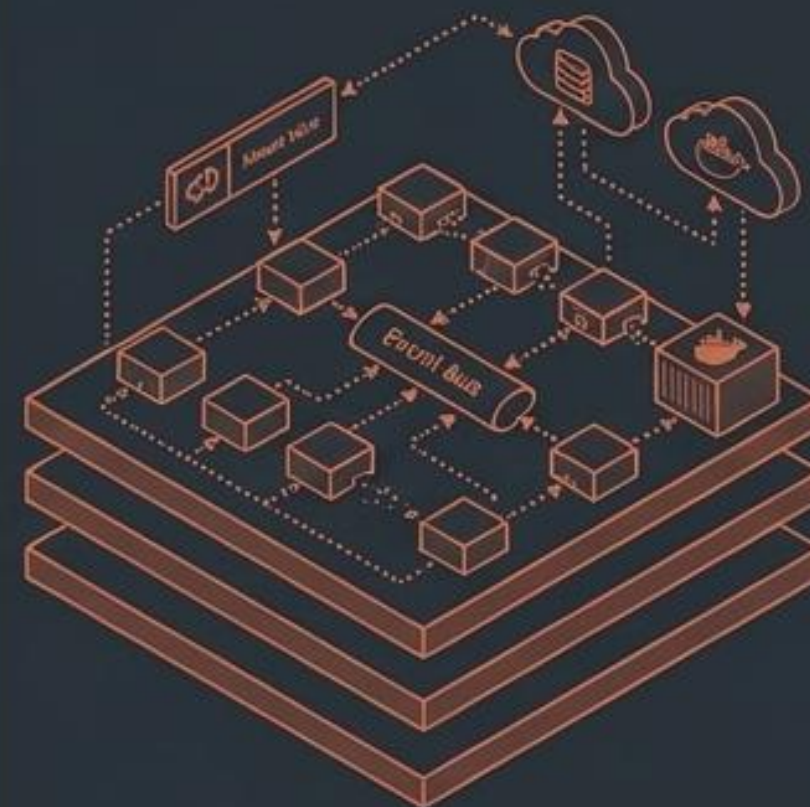
기술: Java, Spring Boot, 기본적인 DB 연결.



Intermediate (쇼핑몰 / 예약 & 관리자 페이지)

목표: 결제 연동, 복잡한 테이블 관계, 권한 분리.

기술: Spring Boot, JPA, Thymeleaf(관리자용) + React/Vue(사용자용).



Advanced (MSA / 채팅 앱)

목표: 실시간 알림, 무상태 인증, 마이크로서비스 구조 설계.

기술: JWT, OAuth, WebSockets, Redis, Docker/AWS.

2026 개발자 역량 우선순위 레이더

